

UNIwersytet WarMińsko-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Kierunek: Informatyka

Dawid Kaczyński

Przetwarzanie obrazu w aplikacji webowej
z wykorzystaniem algorytmu redukcji palety
barw i ditheringu

Praca inżynierska wykonana
w Katedrze Metod Matematycznych Informatyki
pod kierunkiem
dr inż. Bartosza Nowaka

Olsztyn, 2026 rok

UNIVERSITY OF WARMIA AND MAZURY IN OLSZTYN
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Field of Study: Computer Science

Dawid Kaczyński

**Image processing in a web application
using a color palette reduction algorithm
and dithering**

Engineer's Thesis is performed
in University of Warmia and Mazury in Olsztyn
under supervision of
dr inż. Bartosza Nowaka

Olsztyn, 2026

Spis treści

Wstęp	3
Wprowadzenie i tło historyczne	3
Przedmiot i cel pracy	3
Struktura pracy	4
Rozdział 1. Charakterystyka wykorzystanych technologii i narzędzi	5
1.1. Język Python	5
1.2. Framework Flask	5
1.3. Biblioteki przetwarzania obrazu	6
1.3.1. Pillow	6
1.3.2. NumPy	6
1.3.3. Scikit-image	6
1.3.4. Random	7
1.4. Technologie warstwy prezentacji	7
1.4.1. Budowa interfejsu HTML5	7
1.4.2. Komunikacja z Pythonem Jinja2	7
1.4.3. JavaScript	8
1.4.4. CSS	8
Rozdział 2. Algorytmy redukcji barw i ditheringu	9
2.1. Równomierna kwantyzacja kolorów	9
2.1.1. Obliczanie liczby kolorów	10
2.1.2. Wady podejścia równomiernego	12
2.1.3. Kwantyzacja z zastosowaniem szumu	12
2.2. Algorytm Median Cut	14
2.3. Dithering	16
2.3.1. Zasada działania	16
2.3.2. Implementacja i wyzwania	17
2.4. Metryki oceny jakości obrazu	17
2.4.1. Koncepcja SSIM	17
2.4.2. Interpretacja wyników	17
2.4.3. Weryfikacja poprawności redukcji	18
Rozdział 3. Analiza wymagań i projekt systemu	19
3.0.1. Wymagania funkcjonalne	19
3.0.2. Wymagania нефункционалне	19
3.1. Architektura aplikacji	19
3.1.1. Interfejs użytkownika	19
3.1.2. Warstwa przetwarzania po stronie serwera	20
3.1.3. Warstwa logiki obliczeniowej	20
Rozdział 4. Implementacja systemu	21

4.1.	Implementacja interfejsu użytkownika	21
4.1.1.	Strona główna i stylizacja	21
4.1.2.	Elementy formularza sterującego	21
4.1.3.	Przesyłanie danych do serwera	22
4.1.4.	Widok prezentacji wyników	22
4.1.5.	Widok analizy porównawczej	24
4.2.	Implementacja logiki backendu	25
4.2.1.	Zarządzanie strukturą plików i katalogiem statycznym	25
4.2.2.	Obsługa żądań HTTP	25
4.2.3.	Warunkowe przetwarzanie	26
4.2.4.	Mechanizm przekierowania	26
4.3.	Implementacja algorytmów przetwarzania obrazu	26
4.3.1.	Implementacja kwantyzacji równomiernej	26
4.3.2.	Implementacja kwantyzacji uwzględniającej szum	31
4.3.3.	Implementacja algorytmu Median Cut	34
4.3.4.	Implementacja ditheringu (Algorytm Floyda-Steinberga)	39
	Bibliografia	44
	Streszczenie	45
	Abstract	46

Wstęp

Wprowadzenie i tło historyczne

Od początków ery cyfrowej, sposób reprezentacji i przechowywania obrazów stanowił jedno z kluczowych wyzwań informatyki. W czasach pierwszych komputerów osobistych, takich jak ZX Spectrum, Commodore 64 czy wczesne systemy IBM PC, ograniczenia sprzętowe wymuszały drastyczną oszczędność pamięci. Karty graficzne, takie jak CGA (*Color Graphics Adapter*) z 1981 roku, pozwalały na jednoczesne wyświetlanie zaledwie czterech kolorów z palety szesnastu, a późniejszy standard VGA (*Video Graphics Array*) spopularyzował tryb 256 kolorów. W tamtych czasach redukcja barw nie była wyborem artystycznym, lecz koniecznością technologiczną. Programiści i graficy musieli stosować zaawansowane techniki, takie jak dithering (rozpraszanie błędu), aby symulować brakujące odcienie i oszukać ludzkie oko, tworząc iluzję głębi na płaskich, ubogich w kolory wyświetlaczach.

Wraz z rozwojem technologii i upowszechnieniem standardu *True Color* (24-bitowa głębia kolorów), pozwalającego na wyświetlanie ponad 16 milionów barw, problem ograniczonej palety teoretycznie stracił na znaczeniu. Jednakże w dobie Internetu, gdzie szybkość ładowania stron i objętość przesyłanych danych mają kluczowe znaczenie, optymalizacja grafik powróciła do łask. Formaty takie jak GIF czy PNG-8 nadal wykorzystują paletę indeksowaną (ograniczoną do 256 kolorów), aby zapewnić bezstratną kompresję i przezroczystość przy minimalnym rozmiarze pliku. Co więcej, w ostatnich latach obserwuje się renesans estetyki „retro” i *pixel art*, gdzie celowa redukcja kolorów jest środkiem wyrazu artystycznego w grach wideo i projektowaniu interfejsów.

Przedmiot i cel pracy

Niniejsza praca poświęcona jest zagadnieniu kwantyzacji kolorów, czyli procesowi redukcji liczby unikalnych barw w obrazie cyfrowym przy jednoczesnym staraniu o zachowanie jak najwierniejszego podobieństwa do oryginału. Tematyka ta łączy w sobie elementy algorytmiki, przetwarzania sygnałów oraz psychofizjologii widzenia. Głównym wyzwaniem w tym obszarze jest znalezienie balansu między stopniem kompresji (liczbą kolorów) a jakością wizualną, która często degradowana jest przez zjawisko pasmowania (ang. *banding*) – powstawania wyraźnych, nienaturalnych granic między odcieniami.

Głównym celem pracy jest zaprojektowanie i zaimplementowanie interaktywnej aplikacji internetowej, umożliwiającej użytkownikom przeprowadzanie procesu redukcji kolorów na dowolnych obrazach. System ma służyć jako narzędzie analityczne i edukacyjne, pozwalające na bezpośrednie porównanie skuteczności różnych algorytmów kwantyzacji oraz technik ditheringu.

W ramach realizacji celu głównego wyznaczono następujące cele szczegółowe:

1. Implementacja i analiza porównawcza dwóch odmiennych podejść do redukcji barw: statycznej **kwantyzacji równomiernej** oraz adaptacyjnego algorytmu **Median Cut**.
2. Zastosowanie algorytmu **Floyda-Steinberga** w celu zbadania wpływu dyfuzji błędu na subiektywną i obiektywną jakość obrazu wynikowego.
3. Implementacja metryki **SSIM** (*Structural Similarity Index*), pozwalającej na numeryczną ocenę stopnia degradacji obrazu po przetworzeniu.
4. Stworzenie nowoczesnego interfejsu użytkownika w oparciu o technologie webowe, co zapewni uniwersalność i dostępność narzędzia bez konieczności instalacji dodatkowego oprogramowania.

Struktura pracy

Praca została podzielona na cztery rozdziały, które logicznie prowadzą od teoretycznych podstaw, przez analizę narzędzi, aż po praktyczną implementację rozwiązania.

Rozdział pierwszy stanowi przegląd technologii i narzędzi wykorzystanych w procesie tworzenia aplikacji. Omówiono w nim język Python, który pełni rolę silnika obliczeniowego, oraz mikro-framework Flask, odpowiedzialny za obsługę warstwy serwerowej. Przedstawiono również kluczowe biblioteki do przetwarzania obrazu, takie jak *Pillow*, *NumPy* oraz *scikit-image*, uzasadniając ich wybór w kontekście wydajności i dostępnych funkcji. W dalszej części rozdziału scharakteryzowano technologie warstwy prezentacji (frontend), w tym HTML5, CSS oraz silnik szablonów Jinja2.

Rozdział drugi skupia się na teoretycznych aspektach algorytmów redukcji barw. Opisano w nim matematyczne podstawy kwantyzacji równomiernej oraz jej wady wynikające ze sztywnego podziału przestrzeni RGB. Następnie przedstawiono działanie adaptacyjnego algorytmu Median Cut, wyjaśniając mechanizm podziału przestrzeni barw w oparciu o statystykę obrazu. Osobny podrozdział poświęcono technice ditheringu, wyjaśniając zasadę działania macierzy rozpraszania błędu. Rozdział zamyka omówienie metryki SSIM jako narzędzia do weryfikacji jakości postrzeganej przez ludzkie oko.

Rozdział trzeci zawiera analizę wymagań oraz projekt systemu. Zdefiniowano w nim wymagania funkcjonalne i нефункционалне, jakie musi spełniać aplikacja. Przedstawiono architekturę rozwiązania, dzieląc ją na warstwę interfejsu, przetwarzania po stronie serwera oraz logiki obliczeniowej. Rozdział ten stanowi pomost między teorią a praktyczną realizacją projektu.

Rozdział czwarty jest szczegółowym opisem implementacji systemu. Krok po kroku przedstawiono proces tworzenia interfejsu użytkownika, obsługę formularzy oraz mechanizmy komunikacji klient-serwer. Najistotniejszą częścią tego rozdziału jest opis implementacji poszczególnych algorytmów w języku Python, wraz z fragmentami kodu źródłowego odpowiedzialnymi za kluczowe operacje na pikselach. Rozdział kończy prezentacja działającej aplikacji.

Rozdział 1

Charakterystyka wykorzystanych technologii i narzędzi

W projekcie wykorzystano technologie, które są powszechnie stosowane w przetwarzaniu danych oraz tworzeniu aplikacji internetowych. W tej części przedstawiono technologie zastosowane podczas realizacji projektu. Dobór odpowiednich narzędzi jest ważnym etapem podczas projektowania aplikacji informatycznej, ponieważ ma wpływ na jej działanie i możliwości rozwoju. W projekcie zastosowano język Python, który jest często wykorzystywany w przetwarzaniu danych i obrazów. Język ten został wybrany ze względu na dużą liczbę dostępnych bibliotek oraz prostotę tworzenia aplikacji. W dalszej części przedstawiono opis użytych narzędzi oraz ich zastosowanie w zrealizowanym systemie.

1.1. Język Python

Python jest językiem programowania wysokiego poziomu, co oznacza, że jego składnia jest zbliżona do języka naturalnego i łatwa do czytania. Jest także językiem interpretowanym, czyli kod jest wykonywany od razu przez interpreter, bez potrzeby wcześniejszej kompilacji. Python obsługuje dynamiczne typowanie oraz automatyczne zarządzanie pamięcią, co upraszcza tworzenie programów i zmniejsza liczbę błędów. Jego prosta i czytelna składnia ułatwia zrozumienie kodu, a bogata biblioteka standardowa oraz dostępne rozszerzenia pozwalają na tworzenie różnorodnych aplikacji w sposób szybki i efektywny.

Zastosowanie w projekcie: Większość logiki aplikacji została zaimplementowana w języku Python. Odpowiada on za:

- **Implementację algorytmów redukcji kolorów:** W szczególności algorytmu *Median Cut*.
- **Obliczenia matematyczne:** Wykonywanie operacji związanych z kwantyzacją kolorów oraz algorytmami dystrybucji błędu czyli dithering.
- **Zarządzanie systemem plików i konwersję formatów:** Obsługa operacji wejścia-wyjścia.
- **Obsługę komunikacji sieciowej:** Zarządzanie żądaniami HTTP w ramach frameworka Flask oraz sterowanie przepływem danych między przeglądarką użytkownika a logiką aplikacji.

1.2. Framework Flask

Flask to popularny mikroframework do tworzenia aplikacji webowych w języku Python. Oferuje tylko podstawowe funkcje potrzebne do budowy aplikacji, dzięki czemu jest lekki i łatwy w użyciu. Pozwala szybko tworzyć proste strony internetowe oraz obsługiwać żądania HTTP. Flask jest elastyczny, dobrze udokumentowany i wspierany przez dużą społeczność, co ułatwia jego wykorzystanie w praktycznych projektach. [7].

Zastosowanie w projekcie

Flask pełni rolę „kręgosłupa” aplikacji. W pliku `app.py` wykorzystano go do:

- Definiowania ścieżek dostępu (tzw. routing), np. `@app.route("/")`.
- Obsługi przesyłania plików graficznych przez użytkownika (obiekt `request.files`).
- Przekazywania zmiennych obliczeniowych wyniki SSIM, ścieżki do obrazów oraz widoków HTML za pomocą funkcji `render_template`.

1.3. Biblioteki przetwarzania obrazu

Biblioteki przetwarzania obrazu umożliwiają wykonywanie różnych operacji na obrazach cyfrowych. Pozwalają one na wczytywanie i zapisywanie plików graficznych oraz dostęp do pojedynczych pikseli obrazu. Dzięki temu możliwa jest modyfikacja obrazu na poziomie najmniejszych elementów, co pozwala uzyskać oczekiwany efekt końcowy.

1.3.1. Pillow

Biblioteka Pillow w języku Python służy do manipulacji obrazami oraz wykonywania podstawowych operacji przetwarzania obrazu. Obsługuje popularne formaty graficzne, takie jak JPEG czy PNG. Pillow umożliwia wczytywanie, modyfikowanie oraz zapisywanie obrazów, a także wykonywanie operacji takich jak kadrowanie, skalowanie, obracanie czy stosowanie prostych filtrów.

Biblioteka ta jest często wykorzystywana do przetwarzania obrazów. Choć nie należy do najszybszych narzędzi, jest łatwe w użyciu i proste do nauki. Jej możliwości mogą być rozszerzane poprzez integrację z biblioteką NumPy, co pozwala na operacje na poziomie pojedynczych pikseli oraz realizację bardziej zaawansowanych zadań przetwarzania obrazu.[8].

Zastosowanie w projekcie: Biblioteka ta jest podstawowym narzędziem wykorzystywanym do implementacji modułów przetwarzania obrazów.

- Wczytywania obrazów do pamięci operacyjnej `Image.open`.
- Bezpośredniego dostępu do wartości poszczególnych pikseli (`pixels[x, y]`), co było niezbędne przy implementacji własnych pętli.

1.3.2. NumPy

NumPy jest podstawową biblioteką Pythona do obliczeń numerycznych. Umożliwia efektywne operacje na wielowymiarowych tablicach oraz oferuje zestaw funkcji matematycznych do przetwarzania dużych zbiorów danych. Dzięki temu stanowi ważne narzędzie w projektach wymagających intensywnych obliczeń.

Zastosowanie w projekcie: Choć bezpośrednie operacje na pikselach realizowano głównie przez Pillow, NumPy posłużył jako warstwa pośrednia konwertująca obiekty graficzne na macierze liczbowe. Jest to krok niezbędny do działania funkcji obliczających metryki podobieństwa obrazów.

1.3.3. Scikit-image

Scikit-image to zestaw algorytmów do przetwarzania obrazów zbudowany w oparciu o bibliotekę SciPy. Biblioteka ta jest przeznaczona głównie do zastosowań naukowych i badawczych w analizie obrazów [10].

Zastosowanie w projekcie

Biblioteka została wykorzystana do obiektywnej oceny jakości kompresji obrazów. W projekcie użyto funkcji `metrics.structural_similarity` (SSIM), która porównuje obrazy pod względem luminancji, kontrastu oraz struktury. W przeciwieństwie do SSIM, tradycyjny średni błąd kwadratowy (Mean Squared Error - MSE) mierzy jedynie różnicę wartości pikseli między dwoma obrazami, nie uwzględniając przy tym postrzeganej przez człowieka jakości ani struktury obrazu. Dlatego zastosowanie SSIM daje bardziej miarodajne wyniki przy ocenie wizualnej jakości kompresji.

1.3.4. Random

Moduł `random` jest integralną częścią biblioteki standardowej języka Python. Oznacza to, że jest on instalowany automatycznie wraz z interpreterem i nie wymaga pobierania dodatkowych pakietów za pomocą menedżera `pip`. Należy jednak zaznaczyć, że zgodnie z filozofią języka Python, moduł ten nie jest ładowany domyślnie aby skorzystać z jego funkcjonalności, konieczne jest jawne dołączenie go do projektu instrukcją `import random`.

W realizowanym projekcie biblioteka ta została wykorzystana do wprowadzenia elementów stochastycznych w procesie przetwarzania obrazu. Funkcje generujące wartości losowe posłużyły do implementacji techniki „kwantyzacji z szumem”. Celowe wprowadzenie losowych zaburzeń do wartości pikseli przed procesem redukcji kolorów pozwala na rozbitcie regularnych struktur błędów kwantyzacji, co stanowi prostą, lecz skuteczną alternatywę dla bardziej złożonych algorytmów ditheringu.

1.4. Technologie warstwy prezentacji

Interfejs użytkownika został oddzielony od logiki aplikacji. Do budowy strony użyto HTML5, a do wstawiania danych z serwera w szablony wykorzystano Jinja2.

1.4.1. Budowa interfejsu HTML5

W projekcie język HTML wykorzystano do stworzenia szkieletu strony internetowej. Jego najważniejszymi elementami w kontekście działania aplikacji są:

- **Formularz przesyłania plików:** Zastosowano specjalny typ formularza, który umożliwia przesyłanie obrazów z dysku lokalnego użytkownika do aplikacji.
- **Interfejs sterowania:** Użyto znaczników `input` typu `range`, aby stworzyć suwaki. Dzięki nim użytkownik może płynnie zmieniać parametry algorytmów (np. liczbę kolorów) bez konieczności ręcznego wpisywania wartości liczbowych.
- **Układ galerii:** Za pomocą kontenerów `div` podzielono ekran na dwie części, co pozwala na wyświetlenie obrazu oryginalnego i przetworzonego obok siebie w celu łatwego porównania.

1.4.2. Komunikacja z Pythonem Jinja2

Sam HTML jest statyczny i nie może pokazywać wyników obliczeń. Dlatego użyto Jinja2, która pozwala łączyć wygląd strony z logiką aplikacji w Pythonie.

Zastosowanie Jinja2 w projekcie obejmuje trzy główne zadania:

1. **Wstawianie wyników:** Silnik pobiera obliczoną w Pythonie wartość metryki SSIM oraz ścieżkę do przetworzonego obrazu i wstawia je w odpowiednie miejsca w szablonie HTML.
2. **Ukrywanie elementów:** Dzięki instrukcjom `if`, sekcja z wynikami jest ukryta. Dopóki użytkownik nie wgrywa zdjęcia, to nie obliczenia się nie wykonają.

3. **Formatowanie liczb:** Jinja2 automatycznie zaokrągla wyniki liczbowe do czytelnej postaci, co poprawia estetykę interfejsu.

1.4.3. JavaScript

JavaScript to język skryptowy, który pozwala dodawać interaktywne elementy na stronach internetowych.

Zastosowanie w projekcie:

- **Synchronizację suwaków:** Automatyczna aktualizacja pola liczbowego po przesunięciu suwaka wyboru liczby kolorów.
- **Walidację formularza:** Blokowanie przycisku "Convert", gdy nie wybrano pliku.
- **Narzędzie lupy:** Złożony algorytm obsługujący zdarzenia myszy (`mousemove`), który przelicza pozycję kursora i przesuwa tło w oknie podglądu, umożliwiając synchroniczne powiększanie obrazów.

1.4.4. CSS

Arkusze stylów odpowiadają za warstwę wizualną aplikacji.

Zastosowanie w projekcie: Wykorzystano moduł Flexbox do responsywnego rozmieszczenia elementów galerii. Stylizacja obejmuje również niestandardowe tła, efekty przezroczystości oraz efekt rozmycia tła, co nadaje aplikacji nowoczesny wygląd.

Rozdział 2

Algorytmy redukcji barw i ditheringu

Najpopularniejszym sposobem zapisu barw w urządzeniach wyświetlających jest model RGB. Opiera się on na trzech wiązkach światła: czerwonej, zielonej i niebieskiej. Zgodnie z literaturą przedmiotu [1], model ten wynika bezpośrednio z tróchromatycznej teorii widzenia barwnego u człowieka i jest naturalnym standardem dla monitorów komputerowych.

Podstawą zastosowanej w pracy metody redukcji barw jest proces **kwantyzacji równomiernej**. Zgodnie z terminologią przyjętą przez Gonzaleza i Woodsa [2], operacja ta polega na odwzorowaniu ciągłego zakresu wartości intensywności na ograniczony zbiór dyskretnych poziomów N .

W kontekście cyfrowego przetwarzania obrazu, zmniejszenie liczby poziomów kwantyzacji prowadzi do utraty informacji o subtelnych zmianach jasności. Jak wskazują autorzy, skutkuje to powstaniem widocznych, skokowych granic pomiędzy obszarami o zbliżonej barwie. Zjawisko to, degradujące wizualną jakość obrazu poprzez zanik płynnych przejść tonalnych, określane jest w literaturze jako fałszywe konturowanie (ang. *false contouring*).

W celu zminimalizowania tych błędów i lepszego dopasowania palety do rzeczywistej zawartości obrazu w literaturze proponuje się stosowanie algorytmów kwantyzacji adaptacyjnej. Jedną z najczęściej cytowanych metod tego typu jest algorytm przekroju medianowego (ang. Median Cut), wprowadzony przez Paula Heckberta w 1982 roku [3].

W przeciwieństwie do metody równomiernej, która dzieli przestrzeń barw RGB na sztywną siatkę, algorytm Heckberta uzależnia podział od statystyki kolorów występujących w konkretnym obrazie. Proces ten polega na dzieleniu przestrzeni barwnej na mniejsze prostopadłości.

2.1. Równomierna kwantyzacja kolorów

Równomierna kwantyzacja kolorów to najprostsza metoda zmniejszania liczby barw. W standardowym obrazie cyfrowym każdy piksel składa się z trzech kanałów: Czerwonego (R), Zielonego (G) i Niebieskiego (B). Zazwyczaj każdy z tych kanałów może przyjmować jedną z 256 wartości. Metoda równomierna polega na drastycznym ograniczeniu tych wartości poprzez podzielenie każdego kanału na mniejszą, stałą liczbę równych przedziałów i zastąpienie oryginalnych pikseli nowymi wartościami (zob. tabela 2.1). Należy jednak podkreślić, że liczba kolorów obliczona w ten sposób stanowi jedynie wartość maksymalną. Jeżeli obraz wejściowy charakteryzuje się niewielką różnorodnością barw, na przykład jest zdominowany przez jeden odcień, część poziomów kwantyzacji może nie zostać wykorzystana. W rezultacie faktyczna liczba unikalnych kolorów w obrazie wynikowym może być mniejsza niż liczba określona w parametrach kwantyzacji.

Należy zauważyć, że proces redukcji kolorów przeprowadzany jest oddzielnie dla każdego z trzech kanałów obrazu: czerwonego, zielonego i niebieskiego. Ostateczny kolor piksela uzyskuje się dopiero po połączeniu trzech kanałów. Proces ten przedstawiono w.

Tabela 2.1: Granice przedziałów

N	Kolory (N^3)	Przedział wartości wejściowych	Wartość wynikowa
2	8	0 – 127	0
		128 – 255	255
3	27	0 – 63	0
		64 – 191	128
		192 – 255	255
4	64	0 – 42	0
		43 – 127	85
		128 – 212	170
		213 – 255	255

2.1.1. Obliczanie liczby kolorów

Aby określić, ile kolorów ostatecznie pojawi się w obrazie, należy uwzględnić fakt, że każdy kolor powstaje przez połączenie składowych czerwonej (R), zielonej (G) i niebieskiej (B). Liczba kolorów w końcowej palecie K jest zatem iloczynem liczby poziomów wybranych dla każdego z kanałów. Jeśli oznaczymy liczbę poziomów w kanale jako N , wzór można przedstawić następująco:

Jeśli oznaczymy liczbę poziomów jako N , wzór wygląda następująco:

$$K = N \times N \times N = N^3 \quad (2.1)$$

Przykład 1

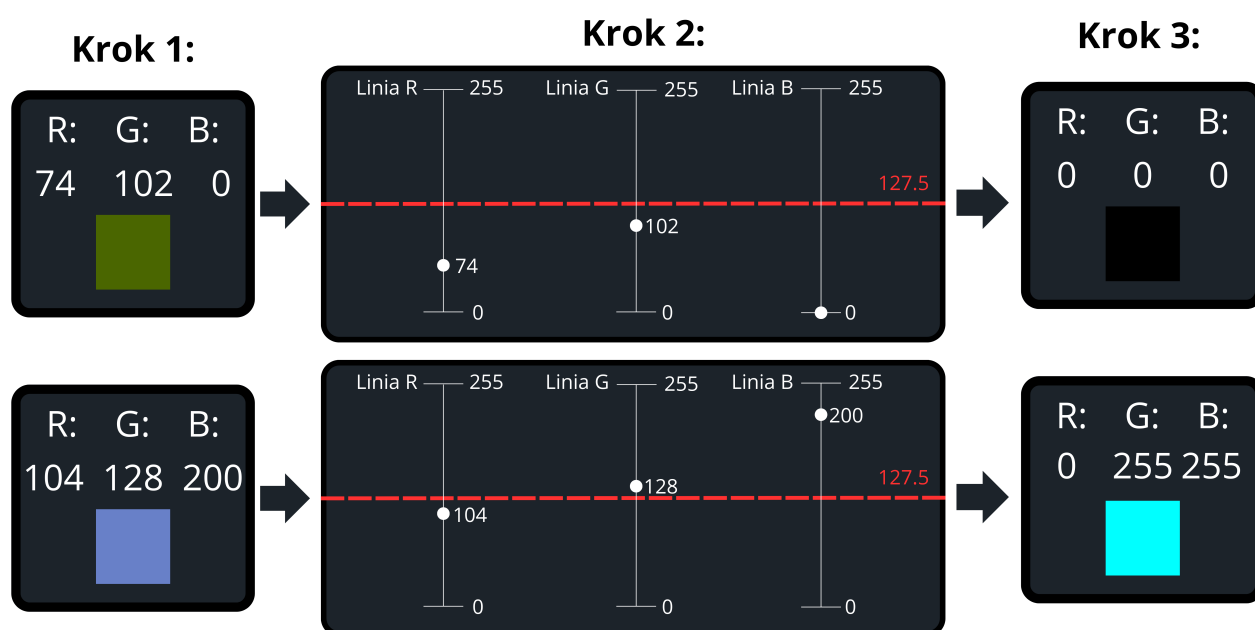
Wybór 2 poziomów na kanał:

- Kanał Czerwony: 2 poziomy
- Kanał Zielony: 2 poziomy
- Kanał Niebieski: 2 poziomy

Ostateczna liczba kolorów w palecie wynosi:

$$2 \times 2 \times 2 = 8 \text{ kolorów}$$

W takim przypadku obraz wynikowy będzie składał się jedynie z podstawowych barw takich jak czarny, biały, czerwony, zielony, niebieski, cyjan, magenta, żółty.



Rysunek 2.1: Schemat procesu równomiernej kwantyzacji kolorów ($N = 2$).

Przykład 2

Wybór 4 poziomów na kanał:

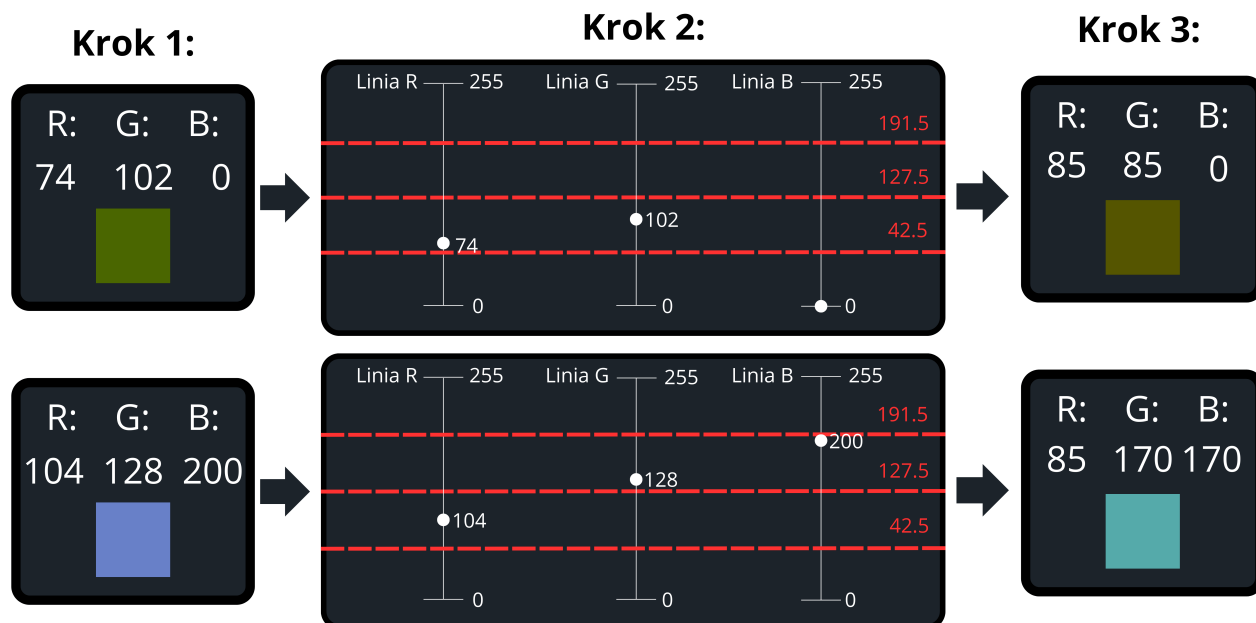
Jeśli zwiększymy precyzję i podzielimy każdy kanał na 4 przedziały, liczba kombinacji znacznie wzrośnie:

- Kanał Czerwony: 4 poziomy
- Kanał Zielony: 4 poziomy
- Kanał Niebieski: 4 poziomy

Ostateczna liczba kolorów w palecie wynosi:

$$4 \times 4 \times 4 = 64 \text{ kolory}$$

Przy 64 kolorach obraz zaczyna być rozpoznawalny, ale wciąż widoczne są wyraźne granice między odcieniami, ponieważ brakuje kolorów.



Rysunek 2.2: Schemat procesu równomiernej kwantyzacji kolorów ($N = 4$).

2.1.2. Wady podejścia równomiernego

Główną wadą tej metody jest jej brak elastyczności. Algorytm traktuje wszystkie kolory w ten sam sposób, niezależnie od zawartości obrazu. Przykładowo, jeśli obraz przedstawia zachód słońca, w którym dominują odcienie czerwieni i pomarańcza, a kolor niebieski występuje rzadko, równomierna kwantyzacja nadal przydziela część palety barw dla odcieni niebieskich. W efekcie część dostępnej palety nie jest wykorzystywana, co obniża jakość odwzorowania kolorów.

2.1.3. Kwantyzacja z zastosowaniem szumu

Podczas standardowej kwantyzacji obrazu może pojawić się niepożądany efekt widocznych pasów kolorów. Efekt ten jest najbardziej widoczny w obszarach obrazu zawierających płynne przejścia jasności. Aby ograniczyć to zjawisko, stosuje się metodę polegającą na dodaniu niewielkiego losowego szumu do wartości pikseli przed wykonaniem kwantyzacji. Dzięki temu piksele o zbliżonych wartościach początkowych nie zawsze są przypisywane do tego samego poziomu jasności. W przypadku braku szumu obszar o jednolitej wartości szarości zostaje zamieniony na jeden kolor, co prowadzi do powstania dużych, jednolitych fragmentów obrazu. Po dodaniu składnika losowego część pikseli trafia do sąsiednich poziomów jasności, tworząc drobną mieszaninę kolorów (zob. rysunek 2.3). Ludzkie oko odbiera taki układ jako płynniejsze przejście tonalne. Chociaż obraz staje się lekko ziarnisty, znikają ostre granice między odcieniami, co poprawia jego naturalny wygląd. W rozwiązaniu do każdego kanału piksela dodawana jest losowa wartość całkowita z zakresu $[-64, 64]$, co pozwala ograniczyć widoczne błędy kwantyzacji przy zachowaniu czytelności obrazu.

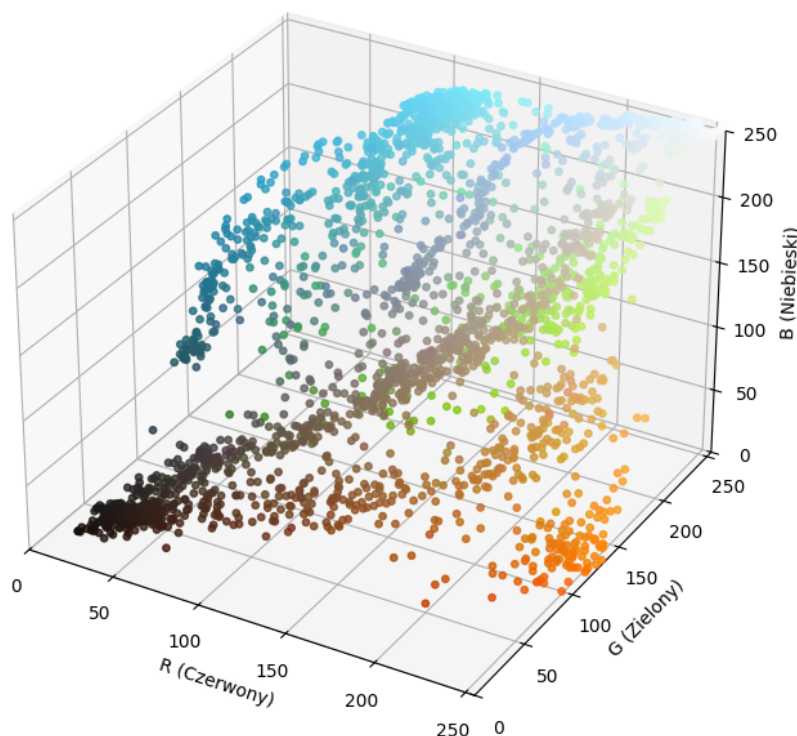


Rysunek 2.3: Schemat procesu równomiernej kwantyzacji kolorów plus szum ($N = 2$).

W efekcie ludzkie oko, które uśrednia kolory sąsiadujących pikseli, odbiera gęstą mieszaninę czarnych i białych punktów jako odcień szarości, mimo że nie występuje on bezpośrednio w paletce barw. Obraz staje się wprawdzie lekko ziarnisty, jednak znikają nienaturalne pasy kolorów, a przejścia są wyraźnie łagodniejsze.

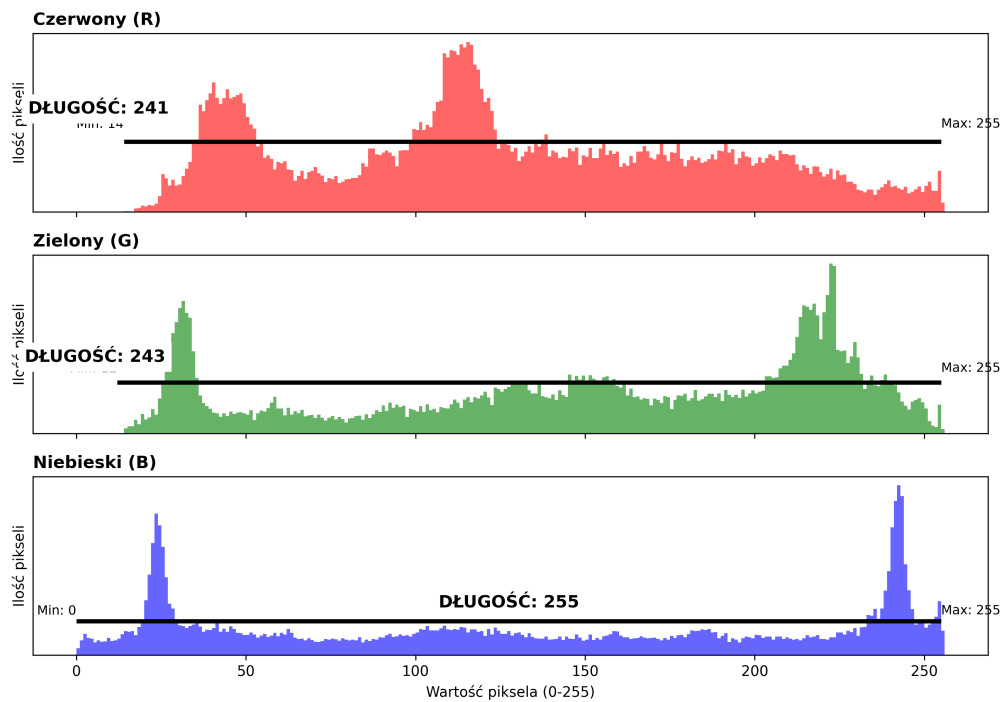
2.2. Algorytm Median Cut

Algorytm Median Cut jest jedną z klasycznych metod kwantyzacji kolorów, czyli redukcji liczby barw obrazu przy zachowaniu możliwie najlepszej percepcyjnej jakości wizualnej. Jego celem jest utworzenie ograniczonej palety kolorów, która w przybliżeniu reprezentuje oryginalną przestrzeń barwną obrazu. Każdy piksel obrazu kolorowego można opisać jako punkt w trójwymiarowej przestrzeni RGB, gdzie osie odpowiadają składowym: czerwonej (R), zielonej (G) oraz niebieskiej (B). W tej reprezentacji każdy piksel traktowany jest jak wektor współrzędnych, a cały obraz stanowi chmurę punktów zawartą wewnątrz sześcianu przestrzeni kolorów (Rysunek 2.4). Na początku algorytmu wszystkie piksele obrazu traktowane są jako jeden zbiór, który można opisać w postaci prostopadłościanu obejmującego pełny zakres występujących wartości RGB.



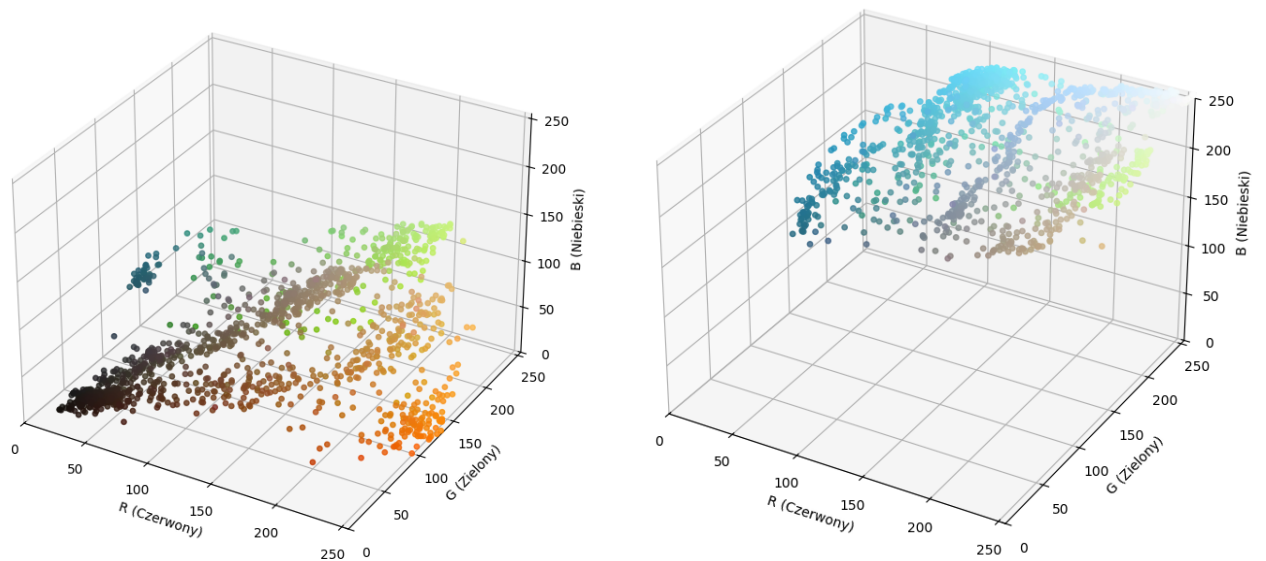
Rysunek 2.4: Trójwymiarowy rozkład przestrzeni barw RGB.

Algorytm działa iteracyjnie, dzieląc przestrzeń barwną na coraz mniejsze obszary, aż do uzyskania zadanej liczby regionów odpowiadających docelowej liczbie kolorów w paletce. W każdej iteracji analizowane są wszystkie aktualnie istniejące obszary (pudełka). Dla każdego z nich wyznaczany jest zakres wartości wzdłuż osi R, G oraz B. Następnie wybierany jest ten obszar, który charakteryzuje się największym rozrzutem w jednej ze składowych barwy. Oznacza to, że zawiera on największe zróżnicowanie kolorystyczne i jego dalszy podział przyniesie największą poprawę jakości odwzorowania barw.



Rysunek 2.5: Wybór osi podziału w algorytmie Median Cut.

Dla wybranego obszaru określana jest oś, wzdłuż której występuje największy zakres wartości (R, G lub B). Piksele należące do tego obszaru są następnie sortowane według wartości wybranej składowej, od najmniejszej do największej. Taki zabieg powoduje, że kolory najbardziej do siebie zbliżone sąsiadują ze sobą w uporządkowanej sekwencji.



(a) Iteracja 1: Podprzestrzeń dolna.

(b) Iteracja 1: Podprzestrzeń górna.

Rysunek 2.6: Podział względem osi Niebieskiej.

Po zakończeniu procesu dzielenia każdy obszar zawiera zbiór pikseli o względnie podobnych barwach. Dla każdego z tych zbiorów wyznaczany jest kolor reprezentatywny, zazwyczaj poprzez obliczenie średniej wartości składowych R, G i B. Otrzymane wartości tworzą końcową paletę kolorów obrazu. Każdy kolor palety stanowi przybliżenie barw zawartych w odpowiadającym mu obszarze przestrzeni RGB.

Tabela 2.2: Statystyki pikseli po pierwszym podziale.

Grupa	Zakres R	Zakres G	Zakres B	Średnia RGB
Część A (dolna)	14–255	14–244	0–127	(116, 100, 57)
Część B (górna)	30–255	98–255	127–255	(130, 199, 207)

Na końcowym etapie algorytmu każdy piksel oryginalnego obrazu jest zastępowany kolorem z wcześniej utworzonej palety. Wybór koloru odbywa się według zasady minimalnej odległości w przestrzeni barw między kolorem piksela a kolorami z palety. Dzięki temu obraz zostaje zredukowany do ograniczonej liczby barw. Algorytm Median Cut redukuje liczbę barw w obrazie, zamieniając płynne przejścia na jednolite obszary kolorów. Kolory dominujące w obrazie są zachowywane, natomiast te mniej występujące mogą zostać uśrednione lub pominięte. Mimo tych ograniczeń, Median Cut jest prostą i skuteczną metodą kwantyzacji kolorów.

2.3. Dithering

W przeciwieństwie do opisanego wcześniej ditheringu losowego, który wprowadza do obrazu niekontrolowany szum, algorytm Floyda-Steinberga opiera się na mechanizmie zwanym dyfuzją błędu. Metoda ta, opracowana w 1976 roku przez Roberta Floyda i Louisa Steinberga, jest powszechnie uznawana za jedną z najlepszych technik redukcji palety barw przy zachowaniu wizualnej jakości obrazu.

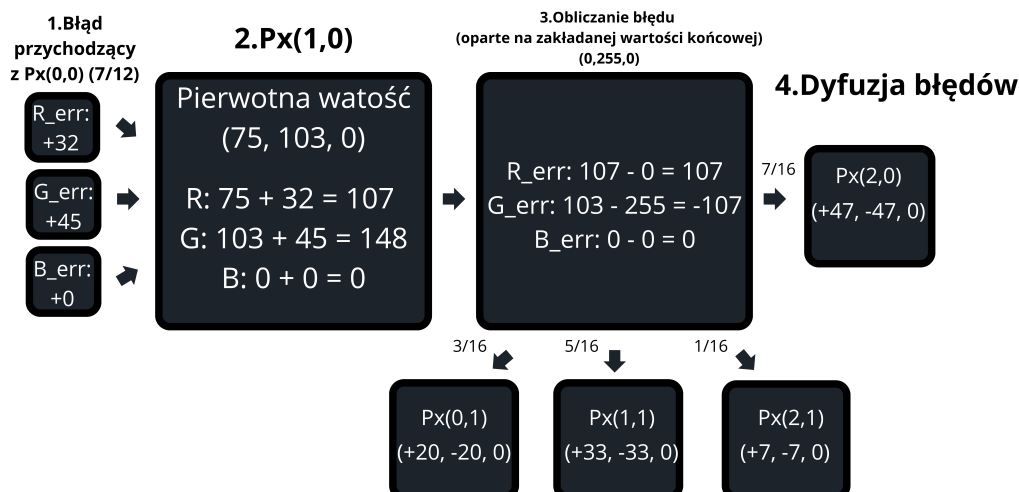
2.3.1. Zasada działania

Algorytm działa tak, że błąd powstały przy kwantyzacji piksela jest przekazywany na nieprzetworzone sąsiednie piksele. Dzięki temu średnia jasność w lokalnym obszarze obrazu jest zachowana, co pozwala zachować wrażenie płynnych przejść nawet przy ograniczonej liczbie kolorów.

Proces dla każdego piksela obrazu przebiega od lewej do prawej i z góry na dół. Dla aktualnego piksela wybierany jest najbliższy kolor z dostępnej palety. Następnie obliczana jest różnica między oryginalnym kolorem a kolorem wybranym. Jest to tzw. błąd kwantyzacji. Obliczony błąd jest dzielony i dodawany do wartości czterech sąsiednich pikseli, które jeszcze nie zostały przetworzone.

Algorytm Floyda-Steinberga [4] wykorzystuje wzorzec wagowy do rozpraszania błędu. Nadmiar lub niedobór wartości jest przekazywany do sąsiada po prawej stronie oraz do trzech sąsiadów w wierszu poniżej. Schemat podziału błędu wygląda w następujący sposób:

- $\frac{7}{16}$ błędu trafia do piksela po prawej stronie $(x + 1, y)$,
- $\frac{3}{16}$ błędu trafia do piksela po lewej na dole $(x - 1, y + 1)$,
- $\frac{5}{16}$ błędu trafia do piksela bezpośrednio na dole $(x, y + 1)$,
- $\frac{1}{16}$ błędu trafia do piksela po prawej na dole $(x + 1, y + 1)$.



Rysunek 2.7: Dyfuzja pojedynczego pixela.

2.3.2. Implementacja i wyzwania

W zaimplementowanym module dyfuzji błędu algorytm przetwarza obraz, dodając błąd do sąsiednich pikseli. Kluczowym elementem implementacji jest zabezpieczenie przed wyjściem wartości pikseli poza dopuszczalny zakres. Ponieważ do sąsiednich pikseli dodawany jest błąd, ich wartość może przekroczyć maksymalną (255) lub spaść poniżej minimalnej (0) wartości dla kanału RGB. W kodzie zastosowano instrukcje warunkowe do ograniczenia wartości pikseli do dozwolonego zakresu, który zapewnia, że zmodyfikowane wartości pozostają w przedziale $[0, 255]$ przed kolejnym krokiem kwantyzacji.

2.4. Metryki oceny jakości obrazu

Proces kwantyzacji i ditheringu wprowadza do obrazu zniekształcenia, takie jak szum czy artefakty. Aby obiektywnie ocenić wpływ tych operacji na jakość obrazu, stosuje się matematyczne miary podobieństwa. Tradycyjne metryki, takie jak błąd średniokwadratowy (MSE) czy szczytowy stosunek sygnału do szumu (PSNR), są łatwe w obliczeniach, ale często słabo odpowiadają subiektywnej ocenie człowieka. Dlatego w niniejszej pracy do oceny jakości zastosowano wskaźnik strukturalnego podobieństwa (ang. *Structural Similarity Index Measure* – SSIM), zaproponowany w 2004 roku przez Z. Wanga i in. [5].

2.4.1. Koncepcja SSIM

Algorytm SSIM porównuje dwa obrazy oryginalny i przetworzony w oparciu o trzy kryteria:

1. **Luminancja:** Porównuje jasność obrazów.
2. **Kontrast:** Ocenia różnice w zakresie zmienności jasności.
3. **Struktura:** Mierzy korelację między obrazami, czyli wzajemne ułożenie pikseli.

2.4.2. Interpretacja wyników

Wartość wskaźnika SSIM mieści się w przedziale $[-1, 1]$, jednak w praktyce przetwarzania obrazów zazwyczaj przyjmuje się wartości z zakresu $[0, 1]$. SSIM równy 1 oznacza, że obrazy są identyczne. Wartości około 0,95–0,99 wskazują na bardzo wysoką jakość, przy której zmiany

są niemal niezauważalne dla oka. Niższe wartości sygnalizują wyraźną degradację struktury obrazu, taką jak rozmycie, silny szum lub widoczne pasy.

2.4.3. Weryfikacja poprawności redukcji

Aby sprawdzić działanie algorytmów kwantyzacji, przygotowano dodatkowy program. Skrypt pobiera wszystkie piksele przetworzonego obrazu i liczy liczbę unikalnych kombinacji kolorów RGB. Narzędzie służy do weryfikacji, czy liczba otrzymanych kolorów nie przekracza ustalonej wartości granicznej K (np. dla $K = 16$ skrypt powinien zwrócić wartość ≤ 16)).

Rozdział 3

Analiza wymagań i projekt systemu

Niniejszy rozdział przedstawia proces projektowania aplikacji. Zdefiniowano w nim wymagania funkcjonalne i нефункционалне, a także opracowano architekturę systemu oraz projekt interfejsu użytkownika.

3.0.1. Wymagania funkcjonalne

- **Wczytywanie obrazów:** Użytkownik może przesłać plik graficzny.
- **Obsługa formatów i normalizacja:** Obraz jest automatycznie konwertowany do przestrzeni barw RGB (24-bit). Zapewnia to ujednoliconą strukturę danych wejściowych dla algorytmów kwantyzacji.
- **Parametryzacja algorytmu:** Możliwość wyboru liczby kolorów w palecie (od 2 do 256) za pomocą suwaka lub wpisanie wartości ręcznie.
- **Zastosowanie szumu:** Możliwość włączenia lub wyłączenia dyfuzji błędu tylko dla równomiernej kwantyzacji kolorów.
- **Podgląd wyników:** Wyświetlanie obrazu oryginalnego i przetworzonego obok siebie wraz z narzędziem lupy.
- **Ocena jakości:** Automatyczne obliczanie i wyświetlanie wskaźnika SSIM.
- **Analiza statystyczna obrazu:** Aplikacja automatycznie oblicza i wyświetla rzeczywistą liczbę unikalnych kolorów występujących zarówno w obrazie źródłowym, jak i wynikowym. Pozwala to na natychmiastową weryfikację skuteczności algorytmu redukcji.

3.0.2. Wymagania нефункционалне

- **Responsywność:** Interfejs aplikacji dostosowuje się do rozmiaru okna przeglądarki (RWD).
- **Intuicyjność:** Obsługa aplikacji nie wymaga specjalistycznej wiedzy technicznej od użytkownika.

3.1. Architektura aplikacji

Architektura systemu została oparta na modelu klient–serwer i podzielona na logiczne warstwy. Taki podział ułatwia utrzymanie oraz rozwój kodu.

Strukturę systemu podzielono na trzy główne warstwy:

3.1.1. Interfejs użytkownika

Warstwa ta stanowi centralny element systemu i odpowiada za komunikację oraz sterowanie jego działaniem. Została zaimplementowana w języku Python z wykorzystaniem frameworka Flask.

- **Technologie:** HTML5, CSS3, JavaScript.
- **Zadania:**

- Renderowanie interfejsu graficznego (GUI) na podstawie szablonów otrzymanych z serwera.
- Obsługa zdarzeń użytkownika (przesuwanie suwaków, wybór plików).
- Walidacja wstępna danych wejściowych (np. sprawdzenie czy wybrano plik).
- Dynamiczna wizualizacja (narzędzie lupy realizowane przez skrypty JS).

3.1.2. Warstwa przetwarzania po stronie serwera

Stanowi punkt centralny systemu, zarządzający komunikacją oraz przepływem sterowania. Została zaimplementowana w języku Python z wykorzystaniem frameworka Flask.

- **Komponent:** Plik `app.py`.
- **Zadania:**
 - Obsługa żądań HTTP (metody GET i POST).
 - Routing, czyli mapowanie adresów URL na konkretne funkcje w kodzie.
 - Weryfikacja przesyłanych plików pod kątem dopuszczalnych formatów obrazów.
 - Przygotowanie widoku interfejsu użytkownika w postaci strony HTML.

3.1.3. Warstwa logiki obliczeniowej

Warstwa ta odpowiada za operacje obliczeniowe związane z przetwarzaniem obrazów, niezależnie od warstwy interfejsu.

- **Komponent:** Moduły `processor.py` oraz biblioteki zewnętrzne (NumPy, Pillow).
- **Zadania:**
 - Normalizacja danych wejściowych (konwersja do macierzy RGB).
 - Realizacja algorytmów redukcji barw (Median Cut, Kwantyzacja).
 - Implementacja algorytmów dystrybucji błędu (Floyd-Steinberg Dithering).
 - Obliczenia SSIM oraz zliczanie unikalnych kolorów.
 - Realizacja operacji na poziomie pojedynczych pikseli.

Rozdział 4

Implementacja systemu

W tym rozdziale opisano proces tworzenia aplikacji. Ze względu na przyjętą architekturę podzielono go na dwie główne części. Pierwsza dotyczy warstwy prezentacji (frontend), obejmującej interfejs graficzny i mechanizmy interakcji z użytkownikiem. Druga część opisuje warstwę serwerową (backend), w której zaimplementowano logikę aplikacji, komunikację z klientem oraz algorytmy przetwarzania obrazów.

4.1. Implementacja interfejsu użytkownika

W tym podrozdziale opisano, jak została stworzona warstwa prezentacji aplikacji frontend. Opis podzielono na etapy, które odzwierciedlają budowę i działanie całego rozwiązania webowego.

4.1.1. Strona główna i stylizacja

Wizualna warstwa aplikacji została zrealizowana przy użyciu kaskadowych arkuszy stylów (CSS). Zastosowano statyczne tło, które nie odwraca uwagi od panelu roboczego. Głównym elementem strukturalnym strony startowej jest centralnie umieszczony kontener (`<div>`), który grupuje wszystkie elementy interaktywne formularza.

Kontener ten pełni rolę „centrum sterowania”, w którym użytkownik definiuje parametry procesu przetwarzania

4.1.2. Elementy formularza sterującego

Wewnątrz głównego kontenera zaimplementowano formularz HTML, który umożliwia interakcję z aplikacją. Składa się on z kilku kluczowych sekcji.

Wczytywanie obrazu

Do przesyłania plików graficznych wykorzystano standardowy element wejściowy typu `file`. Został on skonfigurowany tak, aby akceptować wyłącznie pliki obrazowe (atrybut `accept`). Dla pól numerycznych wprowadzono ograniczenia `min="2"` i `max="255"`, dzięki czemu przeglądarka blokuje wartości spoza dozwolonego zakresu i chroni algorytm przed błędnymi danymi.

Listing 4.1: Pole wyboru pliku

```
<div class="slider-container">
  <input type="range" id="slider" name="slider_value" min="2"
    max="255" value="4">
  <input type="number" id="numberInput" value="4" min="2" max="255">
</div>
```

Wybór liczby kolorów

Aby umożliwić użytkownikowi wybór liczby kolorów w palecie (K), przygotowano dwa sposoby wprowadzania wartości. Użytkownik ma do dyspozycji suwak poziomy (`type="range"`) o zakresie od 2 do 255. Obok suwaka umieszczono pole liczbowe (`type="number"`), które pozwala na ręczne wpisanie precyzyjnej wartości. Oba elementy są ze sobą synchronizowane za pomocą języka JavaScript oraz ustawiona domyślna wartość na cztery.

Listing 4.2: Elementy sterujące wyborem liczby kolorów

```
<div class="slider-container">
  <input type="range" id="slider" min="2" max="255" value="4">
  <input type="number" id="numberInput" value="4" min="2" max="255">
</div>
```

Konfiguracja algorytmów

Dla algorytmu kwantyzacji równomiernej dodano opcję szumu, co redukuje efekt nierównych przejść tonalnych. Funkcjonalność ta jest sterowana za pomocą pola wyboru `checkbox`. Po zaznaczeniu wysyłana jest do serwera dodatkowa wartość logiczna flaga.

Listing 4.3: Pole wyboru opcji szumu

```
<input type="checkbox" id="szum" name="dodaj_szum">
```

4.1.3. Przesyłanie danych do serwera

Zatwierdzenie wprowadzonych ustawień odbywa się za pomocą przycisku typu `submit`. Kliknięcie tego przycisku inicjuje wysłanie formularza metodą `POST` do kontrolera aplikacji Flask (zdefiniowanego w pliku `app.py`). Dzięki atrybutowi `enctype="multipart/form-data"` w nagłówku formularza, możliwe jest poprawne przesłanie zawartości pliku graficznego wraz z parametrami tekstowymi.

Listing 4.4: Przycisk wysyłający formularz

```
<form action="/" method="POST" enctype="multipart/form-data">
  <button type="submit" id="submit-btn" disabled>Convert</button>
</form>
```

4.1.4. Widok prezentacji wyników

Po przetworzeniu danych przez serwer aplikacja wyświetla widok wyników. Podstawą układu graficznego jest centralny kontener, który grupuje wszystkie elementy i centruje je na ekranie, zapewniając spójny wygląd aplikacji. Widok prezentuje obrazy w kolejności od dołu do góry, aby użytkownik mógł je wygodnie przeglądać.

Na szczycie kontenera umieszczono obraz oryginalny. Jest on punktem odniesienia dla wszystkich porównań. Bezpośrednio pod nim wyświetlane są metryki bazowe:

- **Liczba kolorów:** Rzeczywista liczba unikalnych barw w oryginale.
- **SSIM:** Ze względów optymalizacyjnych została ona zdefiniowana w szablonie statycznie jako wartość 1.0. Pozwoliło to wyeliminować zbędne wywołanie funkcji porównującej (obraz porównywany z samym sobą zawsze zwraca wynik 1.0).

Listing 4.5: Wyświetlanie obrazu oryginalnego i metryk

```
<div class="gallery-item-large">
  <a href="{ { url_for('static', filename=obraz_oryginalny) } }"
    target="_blank" rel="noopener noreferrer">
    
  </a>
  <p class="caption">Oryginalny obraz SSIM: 1.00000</p>
  <p class="caption">Ilość kolorów: { { c_org } }</p>
</div>
```

Wiersz 1 porównanie algorytmów bez ditheringu

Poniżej obrazu referencyjnego znajduje się pierwszy wiersz wyników, w którym zestawiono obok siebie dwie metody redukcji barw.

1. **Lewa strona:** Kwantyzacja równomierna. Jeżeli użytkownik zaznaczył w formularzu opcję dodania szumu, tytuł sekcji oraz wyświetlany obraz automatycznie uwzględniają tę modyfikację.
2. **Prawa strona:** Wynik działania algorytmu Median Cut.
Dla każdego z obrazów wyświetlana jest aktualna liczba kolorów (zgodna z parametrem K lub mniejsza) oraz obliczony wskaźnik SSIM.

Listing 4.6: Siatka wyników dla podstawowej kwantyzacji

```
<div class="gallery-item">
  <a href="{ { url_for('static', filename=obraz_kwantyzacja) } }"
    target="_blank" rel="noopener noreferrer">
    
  </a>
</div>
<p class="caption">Redukcja kolorów:</p>
<p class="caption">Ilość kolorów: { { c_kwant } }</p>
<p class="caption">SSIM: { { "{:.5f}".format(kwantyzacja_wynik) } }</p>

<div class="gallery-item">
  <a href="{ { url_for('static', filename=obraz_median_cut) } }"
    target="_blank" rel="noopener noreferrer">
    
  </a>
</div>
<p class="caption">Median Cut { { slider } }</p>
<p class="caption">Ilość kolorów: { { c_median } }</p>
<p class="caption">SSIM: { { "{:.5f}".format(minimalizacja_wynik) } }</p>
```

Wiersz 2 porównanie algorytmów z ditheringiem

Ostatnia sekcja prezentuje wpływ algorytmu dyfuzji błędu (Floyda-Steinberga) na jakość wizualną. Układ jest analogiczny do poprzedniego: po lewej stronie znajduje się wynik kwantyzacji równomierna z ditheringiem, a po prawej wynik Median Cut z ditheringiem.

Elementy nawigacyjne interfejsu

W górnej części interfejsu umieszczono przyciski sterujące, które pozwalają zarządzać działaniem aplikacji. Przycisk powrotu do strony głównej umożliwia szybkie zresetowanie widoku i

wgranie nowych obrazów. Przycisk do podstrony porównawczej przenosi użytkownika do widoku zestawienia wyników, zachowując stan aplikacji. Do poprawnego wyświetlenia tej strony dynamicznie generowany jest adres URL, zawierający nazwy plików wynikowych i parametry algorytmów.

Poniżej przedstawiono fragment kodu szablonu odpowiadający za implementację przycisków nawigacyjnych oraz mechanizm przekazywania parametrów do widoku porównawczego:

```
<a href="/" class="back">Strona główna</a>
<a href="{ url_for(
    'indexv3',
    filename6=obraz_median_cut_dithering.split('/')[ -1 ],
    filename3=obraz_dithering.split('/')[ -1 ],
    c_half=c_half,
    c_dith=c_dith
) }}" class="back" style="top: 70px;">Porównanie
</a>
```

4.1.5. Widok analizy porównawczej

Ostatni widok aplikacji służy do szczegółowej weryfikacji rezultatów działania zaimplementowanych algorytmów. Strona została podzielona na dwie sekcje prezentujące odpowiednio wynik kwantyzacji oraz dithering. Dynamicznie generowany nagłówek informuje użytkownika o finalnej liczbie kolorów w każdym z przetworzonych obrazów.

W celu umożliwienia dokładnej analizy struktury pikseli, zaimplementowano mechanizm tzw. synchronicznej lupy (Listing 4.7). Rozwiązanie to opiera się na języku JavaScript i realizuje następujące zadania:

- Dynamiczne tworzenie elementu wizualnego lupy pod obrazami.
- Obliczanie pozycji kursora względem współrzędnych obrazu.
- Synchronizacja zdarzeń `mousemove` – przesunięcie kursora nad jednym obrazem powoduje adekwatną zmianę widoku powiększenia dla obu grafik jednocześnie.

W warstwie prezentacji (CSS) kluczowe było zastosowanie dyrektywy `image-rendering: pixelated`. Zapobiega ona wygładzaniu krawędzi przez przeglądarkę podczas skalowania obrazu, co pozwala na wierne odwzorowanie pojedynczych pikseli i artefaktów algorytmicznych w oknie podglądu.

Listing 4.7: Fragment funkcji synchronizującej lupy

```
function moveBothLenses(e) {
    // Pobranie pozycji i obliczenie przesunięcia
    var x = pos.x - (lensW / 2);
    var y = pos.y - (lensH / 2);

    // Aktualizacja pozycji soczewek na obu obrazach
    lens1.style.left = x + "px";
    lens1.style.top = y + "px";

    // Przesunięcie tła w oknach podglądu (efekt zoomu)
    result1.style.backgroundPosition = "-" + (x * cx1) +
        "px -" + (y * cy1) + "px";
    result2.style.backgroundPosition = "-" + (x * cx2) +
        "px -" + (y * cy2) + "px";
}
```

4.2. Implementacja logiki backendu

Głównym plikiem sterującym działaniem aplikacji jest skrypt `app.py`. Odpowiada on za konfigurację środowiska, obsługę żądań użytkownika oraz zarządzanie przepływem danych pomiędzy formularzem a algorytmami przetwarzania.

4.2.1. Zarządzanie strukturą plików i katalogiem statycznym

Już na etapie inicjalizacji aplikacji, system dba o poprawną strukturę katalogów niezbędną do przechowywania danych. Zdefiniowana zmienna `UPLOADS_FOLDER` wskazuje na ścieżkę `static/uploads`. Jest to folder statyczny, co oznacza, że pliki w nim zawarte są publicznie dostępne dla serwera WWW i mogą być bezpośrednio wyświetlane w przeglądarce. Za pomocą polecenia `os.makedirs(..., exist_ok=True)` aplikacja automatycznie sprawdza istnienie tego folderu przy każdym uruchomieniu i tworzy go, jeśli nie istnieje. Wszystkie obrazy – zarówno wgrany oryginał, jak i wyniki działania algorytmów (kwantyzacja, dithering) – są zapisywane w tej jednej, centralnej lokalizacji, skąd są później pobierane przez warstwę prezentacji (HTML).

4.2.2. Obsługa żądań HTTP

Za mapowanie adresów URL na funkcje w Pythonie odpowiada dekorator `@app.route`. Funkcja `index`, obsługująca stronę główną, działa na dwóch metodach HTTP:

GET: wyświetla pusty formularz startowy.

POST: uruchamiana, gdy użytkownik przesyła dane, i realizuje główną logikę aplikacji.

W metodzie POST aplikacja pobiera z formularza trzy elementy:

1. Plik graficzny (`request.files`).
2. Wartość liczbową z suwaka (`slider_value`), która jest rzutowana na typ całkowity (`int`) i służy jako parametr dla algorytmów redukcji kolorów.
3. Opcję "Dodaj szum" (`dodaj_szum`), pobieraną z pola typu checkbox.

4.2.3. Warunkowe przetwarzanie

System wykorzystuje instrukcję warunkową, która wybiera sposób przetwarzania obrazu na podstawie decyzji użytkownika. Aplikacja sprawdza, czy zaznaczono opcję dodania szumu: Jeśli tak: generowane jest losowe ziarno (seed), a do przetwarzania obrazu używane są funkcje uwzględniające szum (kwantyzacja z szumem, dithering z szumem).

Jeśli nie: uruchamiane są standardowe funkcje (kwantyzacja bazowa, dithering bazowy).

Niezależnie od wybranej opcji, każdy przetworzony obraz jest natychmiast zapisywany w folderze static/uploads.

Poniższy listing prezentuje implementację omawianej logiki sterującej w języku Python:

Listing 4.8: Warunkowe uruchamianie algorytmów z obsługą ziarna losowości

```
if czy_dodac_szum:
    moj_seed = random.randrange(sys.maxsize)
    obraz_kwantyzac = kwanty_szum(obraz_path, slider_v, seed=moj_seed)
    obraz_dithering = dithe_szum(obraz_path, slider_v, seed=moj_seed)
else:
    obraz_kwantyzacja = kwantyzacja_baza(obraz_path, slider_value)
    obraz_dithering = dithering_baza(obraz_path, slider_value)
```

W analizowanym fragmencie istotne są dwa elementy implementacji. Po pierwsze, zmienna `obraz_path` przechowuje ścieżkę do pliku w katalogu static/uploads, co zapewnia funkcjom przetwarzającym pewny dostęp do zapisanego wcześniej pliku. Po drugie, gdy włączona jest opcja szumu, generowane jest unikalne ziarno losowości `moj_seed`, które przekazywane jest do funkcji kwantyzacji i ditheringu, zapewniając identyczny rozkład szumu dla tego samego obrazu. Dzięki temu różnice w wynikach wynikają tylko z działania algorytmów, a nie z losowości szumu.

4.2.4. Mechanizm przekierowania

Po zakończeniu obliczeń, w tym wyznaczeniu metryk SSIM i policzeniu liczby kolorów, funkcja nie wyświetla od razu widoku wynikowego. Zamiast tego stosuje przekierowanie przy użyciu `redirect` i `urlfor` do nowej ścieżki (`indexv2`). W trakcie przekierowania wszystkie niezbędne dane – nazwy wygenerowanych plików oraz wyniki liczbowe (SSIM, liczba kolorów) – są przesyłane jako parametry w adresie URL metodą GET. Dzięki temu nowa podstrona ma wszystkie informacje potrzebne do wyświetlenia galerii wyników, a odświeżenie strony nie powoduje ponownego przesłania formularza.

4.3. Implementacja algorytmów przetwarzania obrazu

W niniejszym rozdziale opisano szczegóły implementacji algorytmów redukcji liczby barw oraz ditheringu. Wszystkie rozwiązania zostały zaimplementowane w języku Python (wersja 3.12) z wykorzystaniem biblioteki Pillow do operacji na pikselach oraz biblioteki NumPy do przetwarzania danych w postaci macierzy.

4.3.1. Implementacja kwantyzacji równomiernej

W części praktycznej zaimplementowano algorytm kwantyzacji równomiernej w języku Python. Funkcja przyjmuje jako argumenty obraz wejściowy oraz parametr N (w kodzie jako `poziomy`), określający docelową liczbę poziomów dla każdego kanału RGB. Poniższy listing

prezentuje funkcję realizującą to zadanie, która operuje bezpośrednio na pikselach obrazu przy użyciu biblioteki Pillow.

Listing 4.9: Funkcja dokonująca kwantyzacji kolorów

```
def zakres(value):
    return max(0, min(255, int(value)))

def kwantyzacja(obraz_in, ile_kolorow):
    poziomy = ile_kolorow - 1

    obrazek = Image.open(obraz_in).convert("RGB")
    pixels = obrazek.load()
    szerokosc, wysokosc = obrazek.size

    # Iteracja po wszystkich pikselach obrazu
    for y in range(wysokosc):
        for x in range(szerokosc):
            oldR, oldG, oldB = pixels[x, y]

            if poziomy > 0:
                # Przeliczenie wartosci kanalow wg wzoru kwantyzacji
                nowy_r = round(poziomy * oldR / 255) * (255 / poziomy)
                nowy_g = round(poziomy * oldG / 255) * (255 / poziomy)
                nowy_b = round(poziomy * oldB / 255) * (255 / poziomy)
            else:
                nowy_r, nowy_g, nowy_b = 0, 0, 0

            # Zapisanie nowych wartosci z zabezpieczeniem zakresu
            pixels[x, y] = (
                zakres(nowy_r),
                zakres(nowy_g),
                zakres(nowy_b)
            )

    return obrazek
```

Analiza działania algorytmu

W pierwszej fazie działania algorytmu obraz wejściowy jest konwertowany na model barw RGB. Następnie program wczytuje dane obrazu i pobiera jego szerokość i wysokość.

Kluczowym elementem procesu jest iteracja po mapie pikseli. Algorytm przechodzi kolejno przez współrzędne, iterując po szerokości oraz wysokości obrazu. W każdym kroku pętli odczytywane są oryginalne wartości koloru danego piksela. Następnie dla każdej składowej obliczana jest nowa wartość, zgodnie z zaimplementowanym wzorem kwantyzacji (4.1):

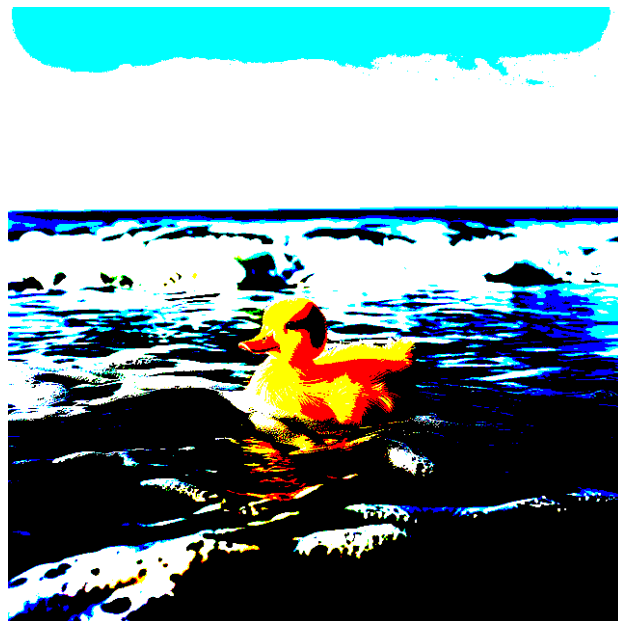
$$C_{new} = \text{round} \left(\frac{N \cdot C_{old}}{255} \right) \cdot \frac{255}{N} \quad (4.1)$$

gdzie C_{new} to nowa wartość, C_{old} to wartość oryginalna, a N oznacza liczbę poziomów.

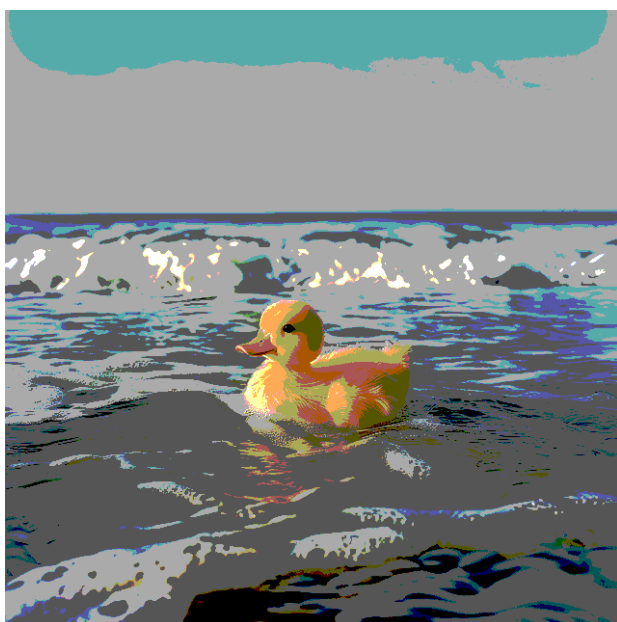
Na samym końcu obliczeń wykorzystywana jest funkcja pomocnicza **zakres**. Jej zadaniem jest zabezpieczenie wyniku, aby ostateczna wartość piksela nie wykroczyła poza dopuszczalny przedział $\langle 0, 255 \rangle$. Tak przetworzone dane są przypisywane jako nowy kolor piksela w obrazie wynikowym.



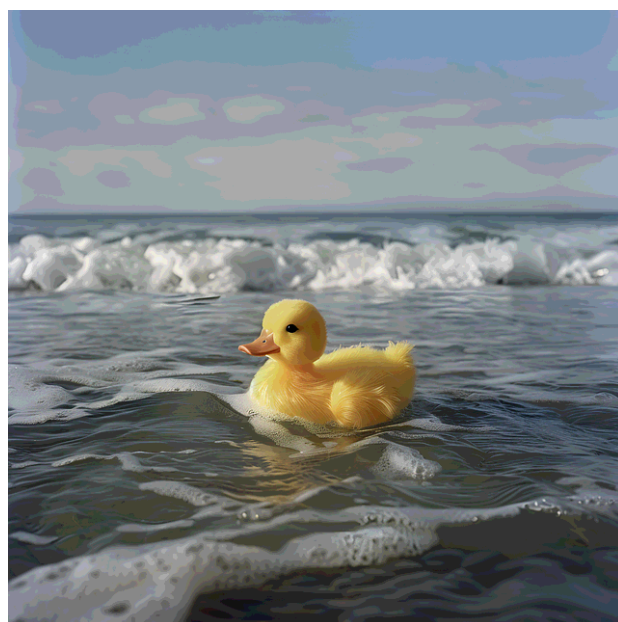
(a) Obraz oryginalny
(74925 kolorów; SSIM = 1.0000)



(b) Kwantyzacja
($N = 2$; 8 kolorów; SSIM = 0.38851)



(c) Kwantyzacja
($N = 4$; 26 kolorów; SSIM = 0.69367)



(d) Kwantyzacja
($N = 16$; 371 kolorów; SSIM = 0.95239)

Rysunek 4.1: Zestawienie wyników działania algorytmu dla różnych parametrów wejściowych.

Analiza oraz wnioski

Rysunek (Rys. 4.1) przedstawia wpływ parametru N na jakość obrazu oraz liczbę dostępnych kolorów. Wraz ze zmniejszaniem wartości N obserwowana jest coraz większa utrata szczegółów obrazu oraz redukcja palety barw. W celu dokładniejszej oceny działania algorytmu przeprowadzono analizę liczbową. Obraz wejściowy charakteryzował się dużą złożonością i zawierał 74 925 unikalnych kolorów. Szczegółowe wyniki eksperymentu zestawiono w tabeli 4.1. Jako

wartość odniesienia przyjęto maksymalną teoretyczną liczbę kolorów równą N^3 , wynikającą z niezależnej kwantyzacji trzech kanałów RGB.

Tabela 4.1: Zestawienie liczby kolorów w zależności od parametru kwantyzacji

Parametr N	Teoretyczne max (N^3)	Uzyskana liczba kolorów
Oryginał	–	74 925
$N = 2$	8	8
$N = 4$	64	26
$N = 16$	4 096	371

Wnioski:

1. **Efektywność redukcji danych:** Analiza wyników pokazuje, że algorytm kwantyzacji równomiernej bardzo skutecznie redukuje liczbę kolorów w obrazie. Dla parametru $N = 16$, mimo że obraz wizualnie nadal przypomina oryginał, liczba unikalnych barw zmniejszyła się z około 75 000 do zaledwie 371.
2. **Wykorzystanie przestrzeni barw ($N=2$):** W przypadku najmniejszej wartości parametru ($N = 2$) uzyskano dokładnie 8 kolorów, co odpowiada teoretycznemu maksimum $2^3 = 8$. Oznacza to, że obraz wejściowy zawierał piksele o skrajnych wartościach w kanałach RGB.
3. **Niejednorodność rozkładu kolorów ($N=4$ i $N=16$):** Dla większych wartości parametru N zauważalna jest wyraźna różnica między liczbą kolorów teoretyczną a faktycznie uzyskaną. Dla $N = 4$ otrzymano 26 kolorów zamiast 64, natomiast dla $N = 16$ jedynie 371 zamiast 4096. Wynika to z charakterystyki analizowanego obrazu, który nie wypełnia równomiernie całej przestrzeni barwnej. Algorytm generuje wyłącznie te kolory, które rzeczywiście występują lub są zbliżone do kolorów obecnych w obrazie wejściowym, co potwierdza nierównomierny rozkład barw w oryginalnej fotografii.
4. **Zanik płynnych zmian odcieni:** Zgodnie z charakterystyką algorytmów redukcji barw opisaną w Rozdziale 2, na obrazie wynikowym wyraźnie widoczne jest zjawisko tzw. *color bandingu*.

W obszarach o naturalnie płynnych gradientach, takich jak niebo czy tafla morza, utracono ciągłość przejść tonalnych. Zamiast gładkiej zmiany barw, obserwujemy wyraźne, odseparowane od siebie pasy jednolitego koloru. Potwierdza to teoretyczne założenie, że prosta kwantyzacja równomierna bez zastosowania algorytmów rozpraszania błędu prowadzi do powstania sztucznych krawędzi i efektu „pikselizacji” w obszarach o niskiej częstotliwości zmian obrazu.

5. **Optymalizacja rozmiaru pliku:** Bezpośrednim skutkiem zmniejszenia liczby unikalnych kolorów jest redukcja rozmiaru pliku wynikowego. Mniejsza różnorodność barw upraszcza strukturę obrazu, co umożliwia skuteczniejszą kompresję i zmniejszenie rozmiaru pliku.
6. **Wpływ kwantyzacji na podobieństwo strukturalne (SSIM):** Zmniejszenie liczby poziomów kwantyzacji prowadzi do istotnego spadku wartości współczynnika SSIM, co oznacza pogorszenie zachowania struktury obrazu względem oryginału.

Poprawa jakości strukturalnej wraz ze wzrostem N : Zwiększanie liczby poziomów kwantyzacji skutkuje wzrostem wartości SSIM, co wskazuje na coraz lepsze odwzorowanie struktur i relacji przestrzennych obecnych w obrazie oryginalnym.

Znaczenie niskich wartości SSIM: Niska wartość SSIM dla małej liczby poziomów kwantyzacji świadczy o silnych zniekształceniach strukturalnych, które są wyraźnie zauważalne wizualnie.

Zachowanie struktury przy wysokim SSIM: Wysoka wartość współczynnika SSIM dla większej liczby poziomów kwantyzacji potwierdza, że obraz wynikowy zachowuje większość kluczowych cech strukturalnych obrazu oryginalnego.

4.3.2. Implementacja kwantyzacji uwzględniającej szum

W celu zniwelowania negatywnego efektu pasmowania (ang. *color banding*), opisanego w wnioskach poprzedniego podrozdziału, zmodyfikowano algorytm poprzez wprowadzenie czynnika losowego. Metoda ta, znana jako dithering losowy, polega na dodaniu do każdego piksela niewielkiej wartości szumu przed wykonaniem właściwej kwantyzacji.

Listing 4.10: Algorytm kwantyzacji z dodaniem szumu losowego

```
import random
def zakres(value):
    """Funkcja ograniczająca wartości do przedziału <0, 255>"""
    return max(0, min(255, int(value)))

def kwantyzacja_z_szumem(obraz_in, ile_kolorow):
    poziomy = ile_kolorow - 1
    obrazek = Image.open(obraz_in).convert("RGB")
    pixels = obrazek.load()
    szerokosc, wysokosc = obrazek.size

    # Iteracja po wszystkich pikselach obrazu
    for y in range(wysokosc):
        for x in range(szerokosc):
            oldR, oldG, oldB = pixels[x, y]

            # Generowanie szumu
            # Losowanie wartości dla każdego kanału
            noiseR = random.randint(-64, 64)
            noiseG = random.randint(-64, 64)
            noiseB = random.randint(-64, 64)

            # Dodanie szumu i zabezpieczenie zakresu
            oldR = zakres(oldR + noiseR)
            oldG = zakres(oldG + noiseG)
            oldB = zakres(oldB + noiseB)

            if poziomy > 0:
                nowy_r = round(poziomy * oldR / 255) * (255 / poziomy)
                nowy_g = round(poziomy * oldG / 255) * (255 / poziomy)
                nowy_b = round(poziomy * oldB / 255) * (255 / poziomy)
            else:
                nowy_r, nowy_g, nowy_b = 0, 0, 0

            pixels[x, y] = (
                zakres(nowy_r),
                zakres(nowy_g),
                zakres(nowy_b)
            )

    return obrazek
```

Analiza wprowadzonych zmian

Względem pierwotnej wersji algorytmu, kod został rozbudowany o moduł generowania wartości pseudolosowych. Zmiana polega na dodaniu do każdej składowej koloru (R, G, B) losowej liczby całkowitej z zakresu $\langle -64, 64 \rangle$. Operacja ta wykonywana jest przed zastosowaniem wzoru kwantyzacji. Dodatkowo wynik sumowania poddawany jest weryfikacji przez funkcję `zakres`, co zapobiega wykroczeniu wartości piksela poza dopuszczalny przedział 0–255.



(a) Obraz oryginalny
(kolorów 74925; SSIM = 1.0000)



(b) Szum + Kwantyzacja
($N = 2$; kolorów 8; SSIM = 0.08082)



(c) Szum + Kwantyzacja
($N = 4$; kolory 52; SSIM = 0.19700)



(d) Szum + Kwantyzacja
($N = 16$; kolorów 2835; SSIM = 0.24417)

Rysunek 4.2: Zestawienie wyników działania algorytmu dla różnych parametrów wejściowych.

Wnioski i analiza porównawcza

Wprowadzenie czynnika losowego wpłynęło nie tylko na wizualny wygląd obrazu, ale również zmieniło statystykę kolorystyczną. Wyniki algorytmu z szumem zestawiono z wynikami algorytmu podstawowego w tabeli 4.2.

Tabela 4.2: Porównanie liczby kolorów.

Parametr N	Teoretyczne Max	Bez szumu	Z szumem (± 64)
Oryginał	–	74 925	74 925
$N = 2$	8	8	8
$N = 4$	64	26	52
$N = 16$	4 096	371	2 835

1. Zwiększenie różnorodności barw ($N=4$ i $N=16$)

Dodanie szumu spowodowało drastyczny wzrost liczby unikalnych kolorów na obrazie wynikowym.

— Dla $N = 4$ liczba kolorów podwoiła się (z 26 do 52). Zbliżyło to wynik do teoretycznego maksimum (64).

— Dla $N = 16$ nastąpił ponad siedmiokrotny wzrost liczby barw (z 371 do 2835).

Mechanizm ten wynika z faktu, że szum modyfikuje wartości pikseli na granicach przedziałów.

2. Brak zmian dla $N=2$

Dla najniższego parametru liczba kolorów pozostała bez zmian (8). Jest to spowodowane bardzo szerokimi przedziałami kwantyzacji (0-127 i 128-255). Szum o amplitudzie ± 64 nie był wystarczająco silny, aby zmusić piksele do "przeskoczenia" do innych, odległych wierzchołków sześciangu RGB.

3. Eliminacja bandingu

Dla $N = 2$ liczba kolorów pozostała bez zmian 8, jednak wizualny odbiór obrazu uległ drastycznej zmianie. Ze względu na dużą odległość między poziomami (np. 0 i 255), dodany szum powoduje gwałtowne przeskoki pikseli między skrajnymi barwami. Wzrost liczby kolorów, widoczny szczególnie przy $N = 16$ skok do 2835 barw, przekłada się bezpośrednio na redukcję efektu pasmowania.

4. Kompromis między rozmiarem pliku a jakością wizualną

Analiza wskazuje na istotną zależność między dodaniem szumu a kompresją danych. Obrazy poddane samej kwantyzacji (bez szumu) zajmują najmniej miejsca na dysku, ponieważ posiadają duże, jednolite obszary łatwe do kompresji. Dodanie szumu zwiększa złożoność obrazu i liczbę kolorów, co nieznacznie zwiększa rozmiar pliku wynikowego. Jest to jednak koszt warty poniesienia mimo lekkiego wzrostu objętości względem wersji bez szumu, obraz nadal jest znacznie mniejszy od oryginału, a jego jakość wizualna i naturalność przejść tonalnych są lepsze.

5. Wpływ szumu na wartość SSIM

Dodanie szumu do obrazu przed kwantyzacją powoduje wyraźny spadek wartości współczynnika SSIM, co oznacza pogorszenie podobieństwa strukturalnego obrazu względem oryginału.

Stabilność SSIM dla losowego szumu

Pomimo losowego charakteru dodanego szumu, wartości SSIM dla tej samej liczby poziomów kwantyzacji nie zmieniają się istotnie pomiędzy kolejnymi próbami.

Wpływ liczby poziomów

Zwiększanie liczby poziomów kwantyzacji powoduje jedynie niewielki wzrost wartości SSIM, ponieważ szum w dużym stopniu zaburza strukturę obrazu.

Porównanie oceny wizualnej z SSIM

Obrazy z dodanym szumem mogą sprawiać wrażenie lepszych wizualnie, jednak osiągają niższe wartości SSIM niż obrazy poddane wyłącznie kwantyzacji.

4.3.3. Implementacja algorytmu Median Cut

W przeciwieństwie do omówionej wcześniej kwantyzacji równomiernej, która dzieli przestrzeń barw w sposób niezależny od treści obrazu, algorytm *Median Cut* należy do grupy metod adaptacyjnych. Jego celem jest taki podział przestrzeni RGB, aby każdy z wygenerowanych kolorów reprezentował zbliżoną liczbę pikseli oryginalnego obrazu.

Etap 1: Iteracyjny podział przestrzeni barw

Sercem algorytmu jest pętla, która dzieli zbiór pikseli na mniejsze podzbiory. W każdej iteracji wybierany jest zbiór o największej rozpiętości tonalnej. Następnie jest on sortowany względem dominującego kanału i dzielony w połowie.

Poniższy fragment kodu odpowiada za wybór osi podziału oraz rozdzielenie pikseli na dwie nowe grupy.

Listing 4.11: Logika podziału zbioru pikseli (Median Cut)

```
# Fragment funkcji median_cut - wewnątrz petli while
# 1. Wyznaczenie rozpietosci dla kanalow R, G, B
r_range = max(r_vals) - min(r_vals)
g_range = max(g_vals) - min(g_vals)
b_range = max(b_vals) - min(b_vals)

# 2. Sortowanie pikseli wzgledem kanalu o najwiekszej rozpietosci
if r_range >= g_range and r_range >= b_range:
    box.sort(key=lambda x: x[0]) # Sortowanie po R
elif g_range >= r_range and g_range >= b_range:
    box.sort(key=lambda x: x[1]) # Sortowanie po G
else:
    box.sort(key=lambda x: x[2]) # Sortowanie po B

# 3. Podzial pudelka w polowie (mediana)
mid = len(box) // 2
boxes.append(box[:mid]) # Lewa czesc
boxes.append(box[mid:]) # Prawa czesc
```

Jak pokazano na rysunku 4.4, algorytm oblicza różnicę $Max - Min$ dla każdego kanału. W tym przypadku największą rozpiętość wykazał kanał czerwony, co zdecydowało kierunek cięcia widoczny na panelu (b) oraz (c). Ostatecznie, z bogatej palety wejściowej uzyskano dwa reprezentatywne kolory uśrednione, które stanowią bazę do kolejnych iteracji podziału.

Etap 2: Generowanie palety wynikowej

Po podziale na żadaną liczbę podzbiorów algorytm wybiera jeden kolor reprezentatywny dla każdego z nich. W zaimplementowanym rozwiązaniu kolor ten jest obliczany jako średnia wartości wszystkich pikseli znajdujących się w danym „pudełku”.

Listing 4.12: Wyznaczanie kolorów palety poprzez uśrednianie

```
palette = []
for b in boxes:
    if len(b) == 0:
        palette.append((0,0,0))
        continue
    # Obliczenie sredniej arytmetycznej dla kazdego kanalu
    r = sum(p[0] for p in b) / len(b)
    g = sum(p[1] for p in b) / len(b)
    bl = sum(p[2] for p in b) / len(b)

    # Dodanie wyznaczonego koloru do palety
    palette.append((int(round(r)), int(round(g)), int(round(bl))))
```

Etap 3: Przypisanie pikseli do najbliższego koloru w paletcie

Na końcu każdy piksel oryginalnego obrazu zostaje zastąpiony kolorem z wygenerowanej palety. Dla minimalizacji błędu wizualnego wybierany jest kolor o najmniejszej odległości euklidesowej w przestrzeni RGB względem oryginalnego piksela.

Listing 4.13: Funkcja wyszukująca najbliższy kolor w paletcie

```
def najblizszy_kolor(pixel, palette):
    r, g, b = pixel
    min_dist = float('inf') # Nieskonczonosc
    best_color = (0, 0, 0)

    for pr, pg, pb in palette:
        # Obliczenie kwadratu odleglosci Euklidesowej
        # (pierwiastkowanie pominieto dla optymalizacji)
        dist = (r - pr) ** 2 + (g - pg) ** 2 + (b - pb) ** 2

        if dist < min_dist:
            min_dist = dist
            best_color = (pr, pg, pb)

    return best_color
```

W celu praktycznej weryfikacji zaimplementowanego algorytmu, przeprowadzono proces kwantyzacji na obrazie testowym. Poniższy rysunek 4.3 prezentuje wizualne efekty działania metody Median Cut dla wybranych wartości docelowej liczby kolorów ($N \in \{2, 4, 16\}$). Zestawienie to pozwala zaobserwować, jak algorytm dobiera reprezentantów barw w zależności od zadanego ograniczenia, starając się zachować kluczowe cechy wizualne oryginału.



(a) Obraz oryginalny
(ok. 75 tys. kolorów; SSIM = 1.0000)



(b) Median Cut
(N = 2; SSIM = 0.67668)



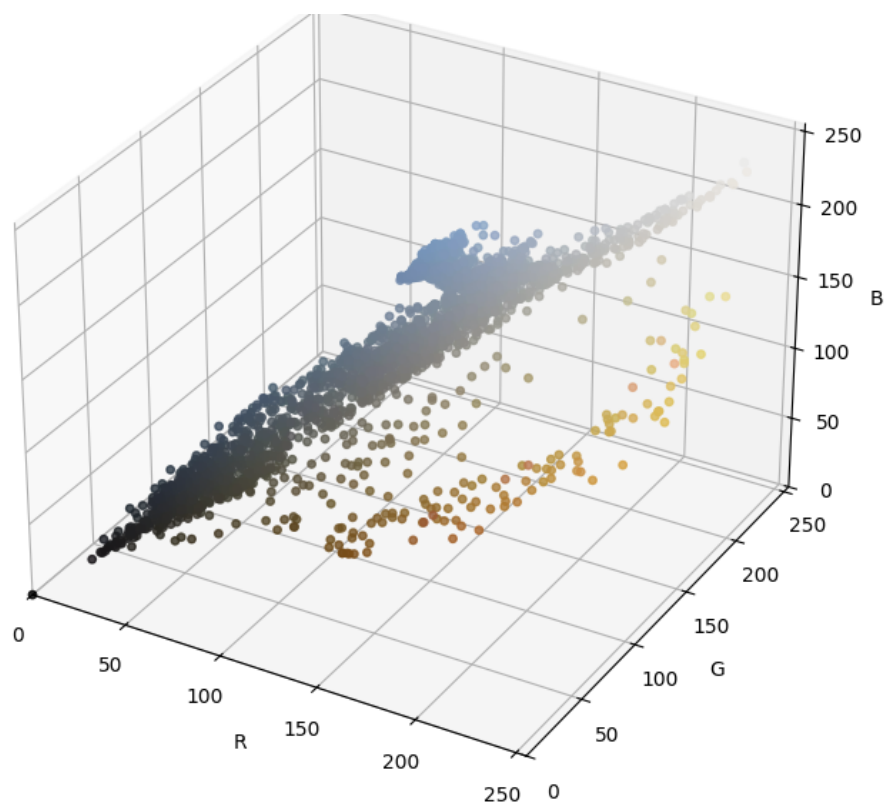
(c) Median Cut
(N = 4; SSIM = 0.74752)



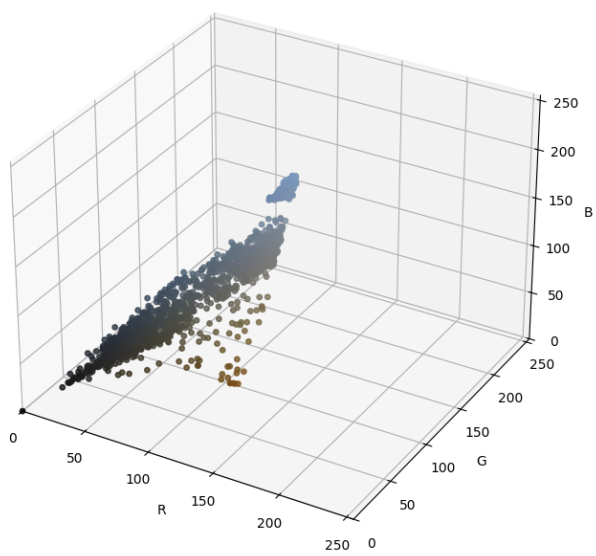
(d) Median Cut
(N = 16; SSIM = 0.85369)

Rysunek 4.3: Zestawienie wyników działania adaptacyjnego algorytmu Median Cut dla różnej liczby kolorów docelowych.

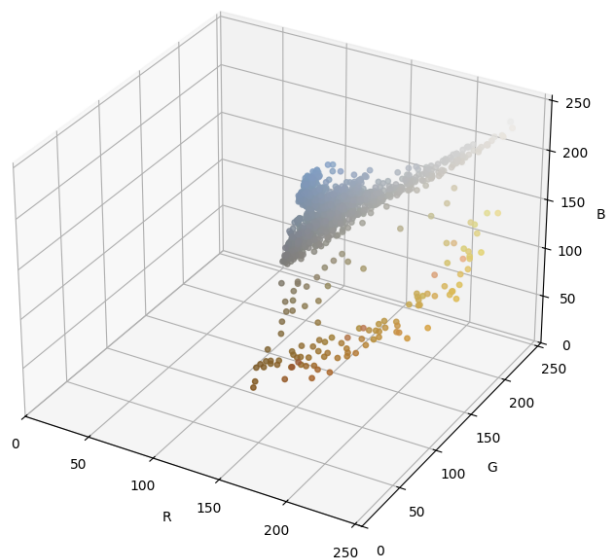
W celu lepszego zrozumienia mechanizmu decyzyjnego przedstawionego na Listingu 4.11, 4.12 oraz 4.13, cały proces algorytmu dla $N = 4$ zilustrowano na rysunku 4.4.



(a) Zbiór wejściowy wszystkie piksele

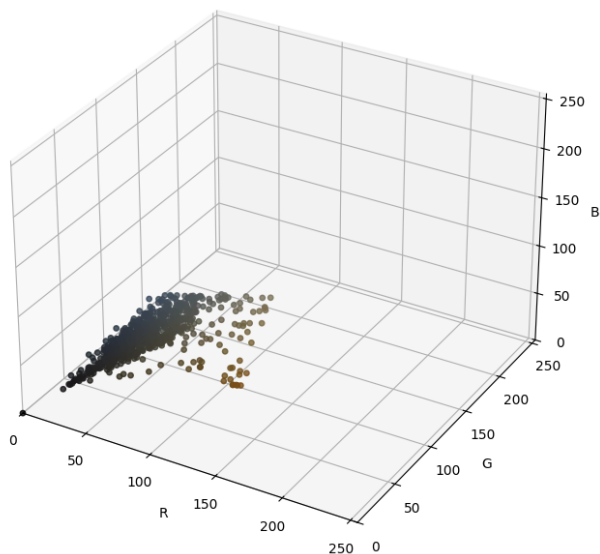


(b) Iteracja 1: Kubełek A

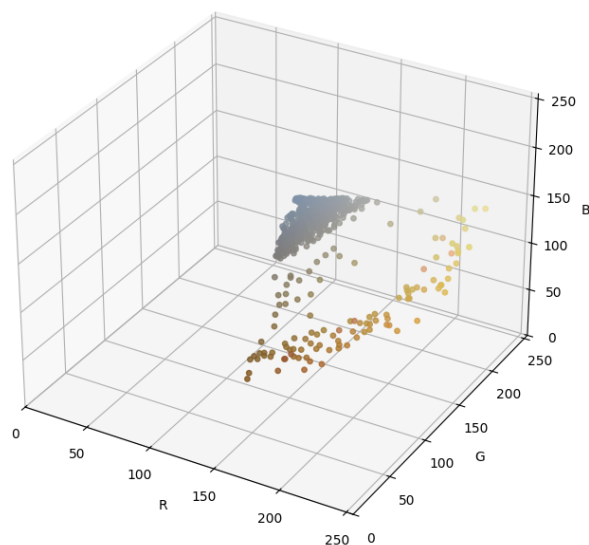


(c) Iteracja 1: Kubełek B

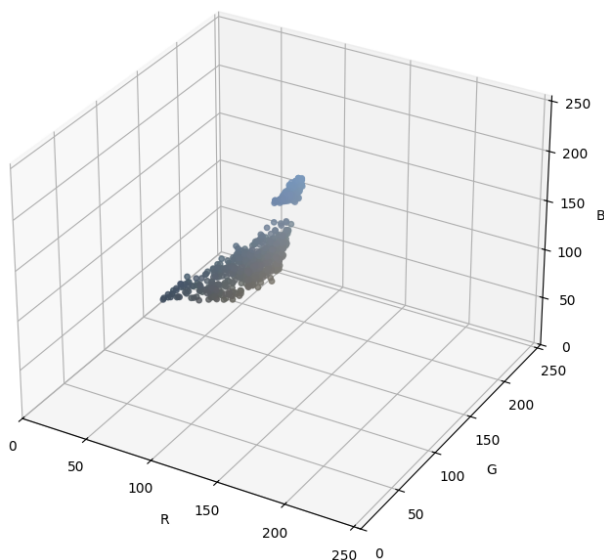
Rysunek 4.4: Wizualizacja procesu Median Cut dla $N=4$. (Ciąg dalszy na następnej stronie)



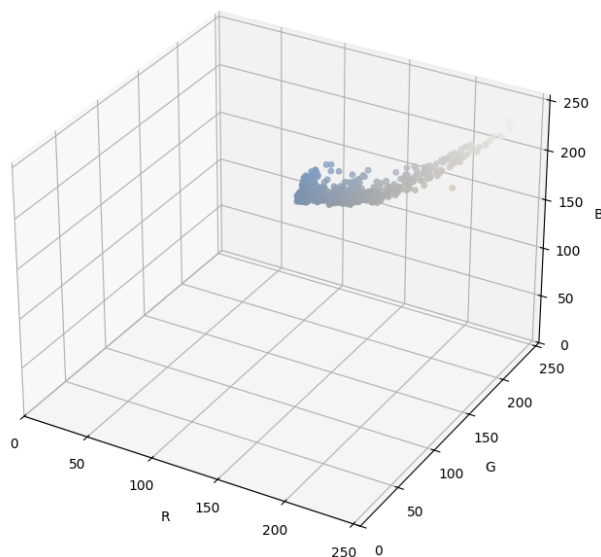
(d) Wynik końcowy 1 (z Kubełka A)



(e) Wynik końcowy 3 (z Kubełka B)



(f) Wynik końcowy 2 (z Kubełka A)



(g) Wynik końcowy 4 (z Kubełka B)

Rysunek 4.4: Finalny podział przestrzeni na 4 obszary barwne.

Wnioski i ocena algorytmu Median Cut

Analiza działania algorytmu *Median Cut* oraz ocena wizualna uzyskanych obrazów pozwalają sformułować następujące wnioski, zwłaszcza w porównaniu z metodą kwantyzacji równomiernej:

1. Odwzorowanie gradientów

Dla małych wartości N odwzorowanie gradientów nie jest idealne, jednak wraz ze wzrostem liczby kolorów jakość wyraźnie się poprawia. Dzięki podziałowi w medianie algorytm *Median Cut* przydziela więcej kolorów obszarom dominującym w obrazie, takim jak niebo i morze. W rezultacie dla większych wartości N przejścia tonalne są znacznie płynniejsze, a efekt pasmowania jest zauważalnie mniejszy w porównaniu z metodą równomierną.

2. Optymalne wykorzystanie palety

W przypadku kwantyzacji równomiernej dla $N = 4$ (teoretycznie 64 kolory) uzyskano jedynie 26 kolorów, co oznacza niewykorzystanie części dostępnej przestrzeni barw. Z kolei w algorytmie *Median Cut* przy zadanej liczbie 16 kolorów otrzymano dokładnie 16 barw, z których każda występuje w obrazie. Oznacza to, że cała paleta jest wykorzystana w sposób efektywny.

3. Jakość obrazu po przetwarzaniu zależy od stopnia jego zróżnicowania kolorystycznego

Algorytm *Median Cut* dąży do minimalizacji błędu średniokwadratowego. W efekcie drobne detale o rzadko występujących kolorach mogą zostać pominięte, jeśli ich udział w obrazie jest niewielki. Przykładem może być mały obiekt o intensywnym kolorze na tle dominującej barwy. Jest to kompromis w porównaniu z metodami o sztywnym podziale przestrzeni barw, które gwarantują zachowanie skrajnych wartości kolorów, takich jak czysta czerwień.

4. Maksymalizacja kompresji danych

Algorytm *Median Cut* wykazuje szczególną użyteczność w zastosowaniach wymagających ekstremalnej redukcji rozmiaru pliku. Pozwala to na zmniejszenie wagi pliku nawet kilkudziesięciokrotnie względem oryginału, zachowując przy tym kluczowe kontury i czytelność kształtów, co jest trudne do osiągnięcia przy zwykłej kwantyzacji równomiernej.

5. Wpływ algorytmu Median Cut na SSIM

Zastosowanie algorytmu Median Cut prowadzi do obniżenia wartości współczynnika SSIM w porównaniu do obrazu oryginalnego, co wynika z redukcji liczby kolorów. Wraz ze wzrostem liczby poziomów kwantyzacji wartość SSIM systematycznie rośnie, co oznacza coraz lepsze zachowanie struktury obrazu. Dla największej liczby poziomów kwantyzacji ($N = 16$) algorytm Median Cut osiąga najwyższą wartość SSIM, co wskazuje na najmniejsze zniekształcenia strukturalne spośród analizowanych przypadków.

4.3.4. Implementacja ditheringu (Algorytm Floyda-Steinberga)

Analiza wcześniejszych wyników pokazała, że sama redukcja liczby kolorów, nawet z użyciem algorytmu *Median Cut*, prowadzi do powstawania efektu pasmowania. W celu ograniczenia tego zjawiska i poprawy jakości wizualnej obrazu zastosowano technikę rozpraszania błędu, opartą na algorytmie Floyda-Steinberga. Metoda ta polega na przenoszeniu błędu kwantyzacji powstałego dla danego piksela na sąsiednie piksele, które nie zostały jeszcze przetworzone. Dzięki temu przejścia tonalne są odbierane jako bardziej płynne.

Macierz dyfuzji błędu

Algorytm Floyda-Steinberga wykorzystuje specyficzną macierz wag do podziału błędu na czterech sąsiadów bieżącego piksela. Dla piksela o współrzędnych (x, y) , błąd jest rozdzielany zgodnie z poniższym schematem:

$$\begin{bmatrix} & * & \frac{7}{16} \\ \frac{3}{16} & \frac{5}{16} & \frac{1}{16} \end{bmatrix} \quad (4.2)$$

Gdzie $*$ oznacza aktualnie przetwarzany piksel. Oznacza to, że:

- $7/16$ błędu trafia do piksela po prawej $(x + 1, y)$,
- $3/16$ błędu trafia do piksela poniżej po lewej $(x - 1, y + 1)$,
- $5/16$ błędu trafia do piksela poniżej $(x, y + 1)$,
- $1/16$ błędu trafia do piksela poniżej po prawej $(x + 1, y + 1)$.

Suma wag wynosi $16/16 = 1$.

Implementacja programowa

Poniższy listing przedstawia implementację algorytmu w języku Python. Funkcja przetwarza piksel po pikselu, modyfikując wartości sąsiadów.

Listing 4.14: Implementacja algorytmu Floyda-Steinberga

```
def floyd_steinberg(obraz_in, paleta):
    img = Image.open(obraz_in).convert("RGB")
    pixels = img.load()
    width, height = img.size

    for y in range(height):
        for x in range(width):
            old_r, old_g, old_b = pixels[x, y]

            # Znalezienie najbliższego koloru i obliczenie błędu
            new_r, new_g, new_b = najblizszy_kolor((old_r, old_g, old_b),
                                                    paleta)
            pixels[x, y] = (new_r, new_g, new_b)

            err_r = old_r - new_r
            err_g = old_g - new_g
            err_b = old_b - new_b

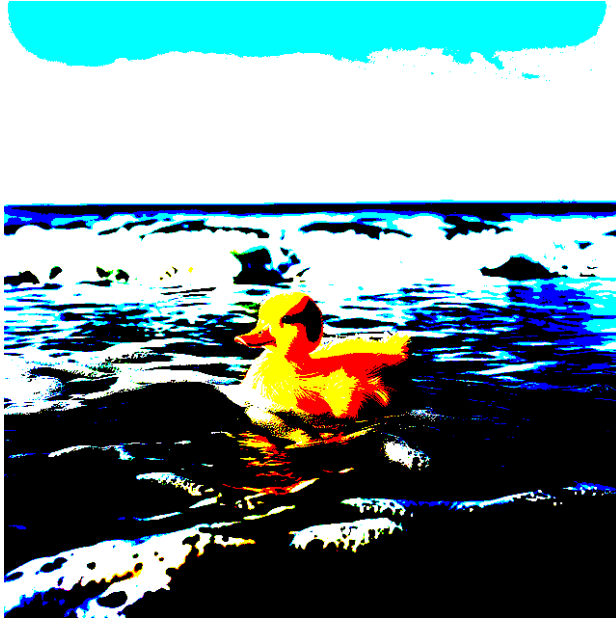
            # Rozproszenie błędu
            # 1. Sasiad Prawy (x+1, y) -> waga 7/16
            if x + 1 < width:
                r, g, b = pixels[x + 1, y]
                pixels[x + 1, y] = (
                    zakres(r + err_r * 7 / 16),
                    zakres(g + err_g * 7 / 16),
                    zakres(b + err_b * 7 / 16)
                )

            # Pozostali sasiadzi (Dolny-Lewy, Dolny, Dolny-Prawy)
            # Aktualizacja wykonywana analogicznie dla wag:
            # 3/16, 5/16, 1/16
            # z uwzględnieniem warunków brzegowych (if ...)
            # ...

    return img
```

Po omówieniu implementacji, kluczowym etapem jest weryfikacja skuteczności algorytmu w praktyce. W tym celu przeprowadzono testy porównawcze na obrazie charakteryzującym się dużą liczbą płynnych przejść tonalnych.

Poniższe zestawienie (Rysunek 4.5) prezentuje bezpośredni wpływ zastosowania dyfuzji błędu na jakość wizualną. Po lewej stronie widoczny jest obraz poddany surowej kwantyzacji, gdzie wyraźnie odznaczają się granice między kolorami. Po prawej stronie przedstawiono ten sam obraz przetworzony algorytmem Floyda-Steinberga, co pozwala ocenić, w jakim stopniu rozpraszanie błędu niweluje niepożądany efekt pasmowania.



(a) Kwantyzacja Równomierna
SSIM: 0.38851



(b) Równomierna + Dithering
SSIM: 0.04802



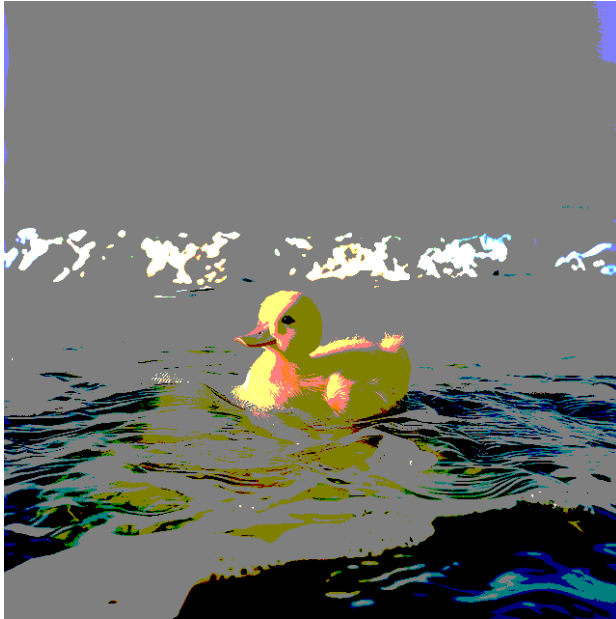
(c) Median Cut
SSIM: 0.81168



(d) Median Cut + Dithering
SSIM: 0.51958

Rysunek 4.5: Zestawienie wyników dla redukcji do 8 kolorów. Dzięki wyrównaniu liczebności palety, widoczna staje się fundamentalna przewaga algorytmu *Median Cut* (dół) nad sztywną siatką kwantyzacji równomiernej (góra). W przypadku metody równomiernej, zastosowanie ditheringu (b) skutkuje powstaniem silnego, ziarnistego szumu, co wynika z konieczności maskowania dużych błędów barwnych. Z kolei dla algorytmu *Median Cut* (d), dithering działa znacznie lepiej skutecznie niweluje pasmowanie i tworzy płynne przejścia, nie wprowadzając przy tym agresywnych zakłóceń struktury obrazu.

W drugim etapie testów zwiększono paletę (Rysunek 4.6). Taka liczba barw pozwala na odwzorowanie kształtów, jednak bez ditheringu nadal widoczny jest wyraźny efekt pasmowania.



(a) Kwantyzacja Równomierna
 $N = 3$; kolorów 15; SSIM: 0.59502



(b) Równomierna + Dithering
 $N = 3$; kolory 24; SSIM: 0.19460



(c) Median Cut
 $N = 15$; kolorów 15; SSIM: 0.85238



(d) Median Cut + Dithering
 $N = 24$; kolory 24; SSIM: 0.65278

Rysunek 4.6: Zestawienie wyników

Analiza wyników dla małej liczby przedziałów ($N = 3$)

Szczegółowa analiza przypadków (a) i (b) ujawnia istotne ograniczenia metody równomiernej oraz specyfikę metryki SSIM:

— **Nieefektywność palety w kwantyzacji równomiernej**

Dla parametru $N = 3$ teoretyczna liczba barw powinna wynosić 27 ($3 \times 3 \times 3$). W praktyce jednak obraz wynikowy (a) składa się zaledwie z 15 kolorów. Wynika to ze sztywnego podziału przestrzeni RGB – wiele zdefiniowanych przedziałów okazało się pustych, ponieważ oryginalny obraz nie zawierał pikseli o takich wartościach. W rezultacie zmarnowano potencjał palety, co skutkuje bardzo słabą jakością wizualną i widocznymi, nienaturalnymi plamami barwnymi.

— **Konflikt oceny wizualnej z metryką SSIM**

Porównanie przypadków (a) i (b) stanowi doskonały przykład rozbieżności między matematyczną oceną podobieństwa a ludzką percepcją. Obraz (a) posiada wyższy wskaźnik SSIM (0.59502), mimo że wizualnie prezentuje się bardzo niekorzystnie. Zastosowanie ditheringu w przypadku (b) znacząco poprawia subiektywny odbiór zdjęcia – przejścia tonalne stają się płynniejsze. Mimo to, wskaźnik SSIM drastycznie spada do poziomu 0.19460. Dzieje się tak, ponieważ SSIM interpretuje wprowadzony szum ditherujący jako błąd, karząc algorytm za zmiany wartości pikseli, które dla ludzkiego oka są korzystnym zabiegiem.

— **Przewaga metody Median Cut**

W zestawieniu z metodą równomierną algorytm *Median Cut* (c) przy tej samej liczbie kolorów (15) osiąga wskaźnik SSIM na poziomie 0.85238. Jest to wynik o klasę lepszy od kwantyzacji równomiernej (0.59502), co potwierdza, że dobór palety pozwala zachować kluczowe informacje o obrazie nawet przy bardzo silnej kompresji.

Podsumowanie i ocena techniki Ditheringu

Analiza obrazów pokazuje, że zastosowanie algorytmu Floyda-Steinberga jest bardzo ważne przy niskiej liczbie kolorów. Dithering zamienia widoczne pasma kolorów na dobre przejścia, których ludzkie oko odbiera jako bardziej naturalne przejścia tonalne. Trzeba jednak pamiętać o ograniczeniach tej metody. Szum może lekko rozmywać drobne szczegóły obrazu i zwiększać trudność kompresji, co czasami prowadzi do większego rozmiaru pliku. Mimo to, łączenie algorytmu *Równomiernej kwantyzacji* oraz *Median Cut* z dyfuzją błędu daje dobry kompromis. Obrazy zachowują płynne przejścia i atrakcyjny wygląd, nawet przy ograniczonej liczbie kolorów.

Bibliografia

- [1] Ryszard Tadeusiewicz, Przemysław Korohoda, *Komputerowa analiza i przetwarzanie obrazów*, Wydawnictwo Fundacji Postępu Telekomunikacji, Kraków 1997.
- [2] Rafael C. Gonzalez, Richard E. Woods, *Digital Image Processing, 3rd Edition*, Prentice Hall, Upper Saddle River, NJ 2008.
- [3] Paul S. Heckbert, “Color Image Quantization for Frame Buffer Display”, *Computer Graphics (SIGGRAPH '82 Proceedings)*, ACM, 1982.
- [4] Robert W. Floyd, Louis Steinberg, *An Adaptive Algorithm for Spatial Grey Scale*, Proceedings of the Society for Information Display, vol. 17, no. 2, 1976, s. 75–77.
- [5] Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, *Image quality assessment: from error visibility to structural similarity*, IEEE Transactions on Image Processing, vol. 13, no. 4, APRIL 2004.
- [6] Paul Heckbert, *Color image quantization for frame buffer display*, Computer Graphics (SIGGRAPH '82 Proceedings), vol. 16, no. 3, 1982, s. 297–307.
- [7] Pallets Projects, *Flask Documentation (Version 3.0.x)*, 2024.
<https://flask.palletsprojects.com/>
- [8] Jeffrey A. Clark et al., *Pillow (PIL Fork) Documentation*, 2024.
<https://pillow.readthedocs.io/en/stable/>
- [9] Python Software Foundation, *random — Generate pseudo-random numbers*, Python 3.12 Documentation.
<https://docs.python.org/3/library/random.html>
- [10] Stéfan van der Walt et al., *scikit-image: Image processing in Python*, PeerJ, vol. 2, 2014.
<https://scikit-image.org/>

Streszczenie

Niniejsza praca dyplomowa porusza problematykę cyfrowego przetwarzania obrazów w kontekście redukcji liczby kolorów kwantyzacji oraz technik rozpraszania błędów kwantyzacji ditheringu. Głównym celem pracy było zaprojektowanie i zaimplementowanie interaktywnej aplikacji internetowej, umożliwiającej użytkownikom przeprowadzanie eksperymentów z różnymi metodami redukcji barw oraz analizę ich wpływu na jakość wizualną obrazu.

W części teoretycznej omówiono matematyczne podstawy reprezentacji barw oraz algorytmy przetwarzania rastrowego. W części praktycznej zaimplementowano środowisko testowe oparte na języku Python i mikro-frameworku Flask, wykorzystujące biblioteki numeryczne NumPy oraz graficzne Pillow. W ramach projektu zaimplementowano i porównano dwie metody kwantyzacji: statyczną kwantyzację równomierną oraz adaptacyjny algorytm *Median Cut*. Dodatkowo, w celu niwelowania efektu pasmowania (*banding*), zaimplementowano algorytm rozpraszania błędów Floyda-Steinberga.

Przeprowadzone testy, obejmujące zarówno ocenę subiektywną, jak i obiektywną z wykorzystaniem metryki SSIM. Wykazano, że zastosowanie ditheringu znacząco poprawia percepcję ciągłości tonalnej, choć odbywa się to kosztem zwiększenia entropii obrazu. Powstała aplikacja stanowi funkcjonalne narzędzie dydaktyczne i analityczne w dziedzinie grafiki komputerowej.

Abstract

This thesis addresses the issue of digital image processing in the context of color reduction quantization and error diffusion techniques dithering. The main objective of the thesis was to design and implement an interactive web application that allows users to experiment with different color reduction methods and analyze their impact on the visual quality of the image.

The theoretical part discusses the mathematical foundations of color representation and raster processing algorithms. The practical part implements a test environment based on Python and the Flask micro-framework, using numerical NumPy and graphics Pillow libraries. As part of the project, two quantization methods were implemented and compared: static uniform quantization and the adaptive Median Cut algorithm. Additionally, in order to eliminate the banding effect, the Floyd-Steinberg error diffusion algorithm was implemented.

The tests carried out included both subjective and objective evaluation using the SSIM metric. It was shown that the use of dithering significantly improves the perception of tonal continuity, although this comes at the cost of increased image entropy. The resulting application is a functional teaching and analytical tool in the field of computer graphics.